

Data Clustering Task Report

목차:

1. 서론

2. EDA

2.5. 기존 주제의 문제점

3. K-Means

4. DBSCAN

5. Hierarchical Clustering

6. 결론

1. 서론

UR3 Cobotops 데이터셋의 센서 데이터를 활용하여 작업 조건에 따른 데이터의 군집 구조를 분석한다.

K-Means, DBSCAN, Hierarchical Clustering을 적용하여 데이터가 자연스럽게 어떠한 그룹으로 분리되는지 확인하고, 군집별 특성을 분석한다.

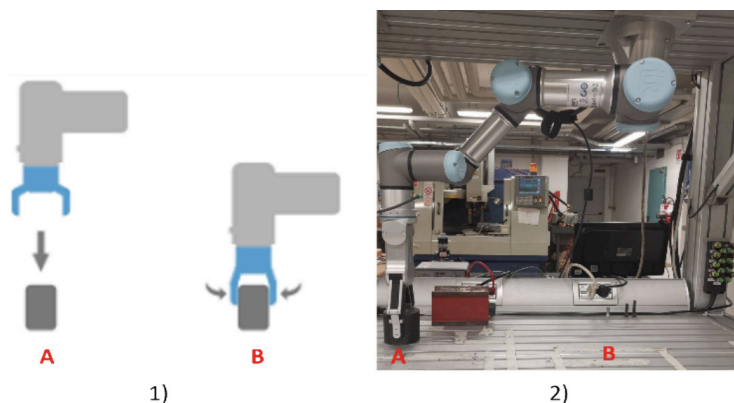
기존의 주제는 Tool Current와 다른 피처들이 물건 하중을 군집할 수 있는지 확인하는 것이었으나 치명적인 문제점이 발견하여 아래에 기술한다.

여러 센서 피처를 활용하여 데이터의 잠재적인 군집 구조를 탐색하고, 군집이 로봇의 운영 상태를 어떻게 나누는지 확인한다.

모든 과정은 **Standard Scaling**을 진행한다.

2. EDA

이 데이터셋을 모은 논문([1] M. Tyrovolas, C. Stylios, K. Aliev, and D. Antonelli, "Leveraging Information Flow-Based Fuzzy Cognitive Maps for Interpretable Fault Diagnosis in Industrial Robotics," in Technological Innovation for Human-Centric Systems, L. M. Camarinha-Matos and F. Ferrada, Eds. Cham, Switzerland: Springer, 2024, pp. 98–110.)을 보면, 이 UR3 Cobotops dataset을 만든 정확한 과정은 다음과 같다.



로봇에게 A에서 B로 물건을 옮기는 작업을 시키며 3가지 조건을 변경했다.

첫번째로, 적재 하중을 1kg, 2kg, 3kg.

두번째로, 이동 속도를 60%, 80%, 100% (최고 210 rad/s 기준).

세번째로, 파지력 (악력)을 80N, 100N, 120N.

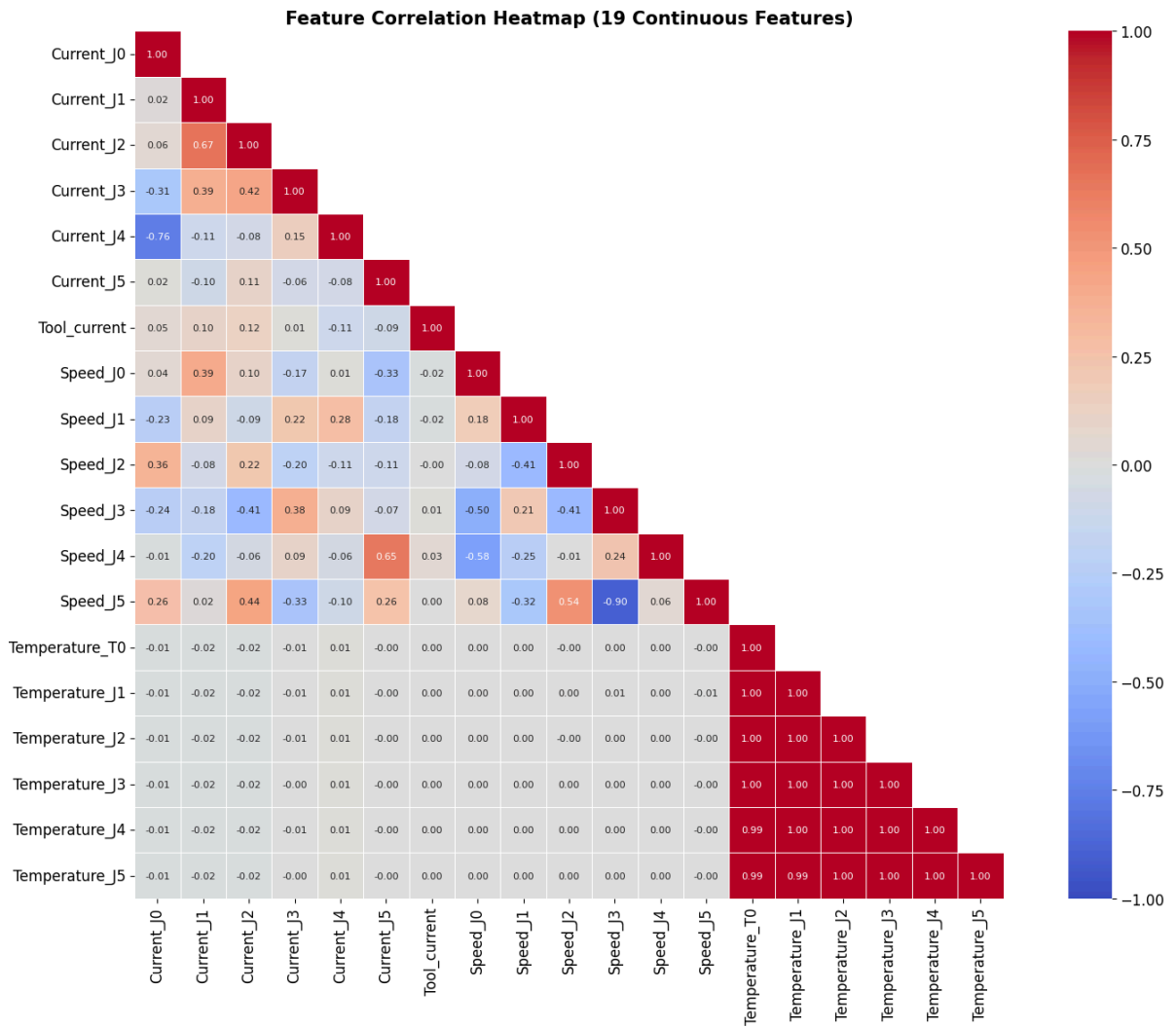
으로 총 27개의 조합을 각 조건마다 25번씩 반복하며 125 Hz로 1초에 125번씩 수집하였다.

#	Column	Non-Null Count	Dtype
0	Num	7409 non-null	int64
1	Timestamp	7409 non-null	str
2	Current_J0	7363 non-null	float64
3	Temperature_J0	7355 non-null	float64
4	Current_J1	7355 non-null	float64
5	Temperature_J1	7355 non-null	float64
6	Current_J2	7355 non-null	float64
7	Temperature_J2	7355 non-null	float64
8	Current_J3	7355 non-null	float64
9	Temperature_J3	7355 non-null	float64
10	Current_J4	7355 non-null	float64
11	Temperature_J4	7355 non-null	float64
12	Current_J5	7355 non-null	float64
13	Temperature_J5	7355 non-null	float64
14	Speed_J0	7355 non-null	float64
15	Speed_J1	7355 non-null	float64
16	Speed_J2	7355 non-null	float64
17	Speed_J3	7355 non-null	float64
18	Speed_J4	7355 non-null	float64
19	Speed_J5	7355 non-null	float64
20	Tool_current	7355 non-null	float64
21	cycle	7409 non-null	int64
22	Robot_ProtectiveStop	7355 non-null	object
23	grip_lost	7409 non-null	bool

데이터셋의 모든 피쳐들이다.

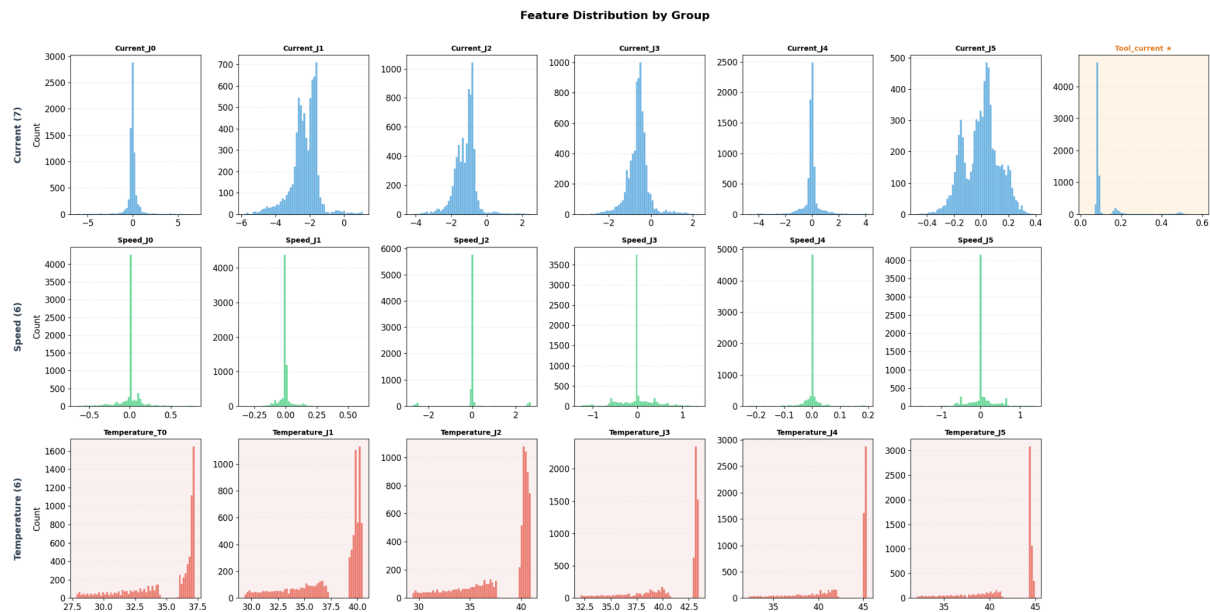
6개 관절의 전류, 온도, 속도 데이터와 그리퍼의 전류가 있다.

이 피쳐들과 파생 변수들이 어떻게 로봇의 운영 상태를 군집화 할 수 있는지 알아보는것이 목표이다.

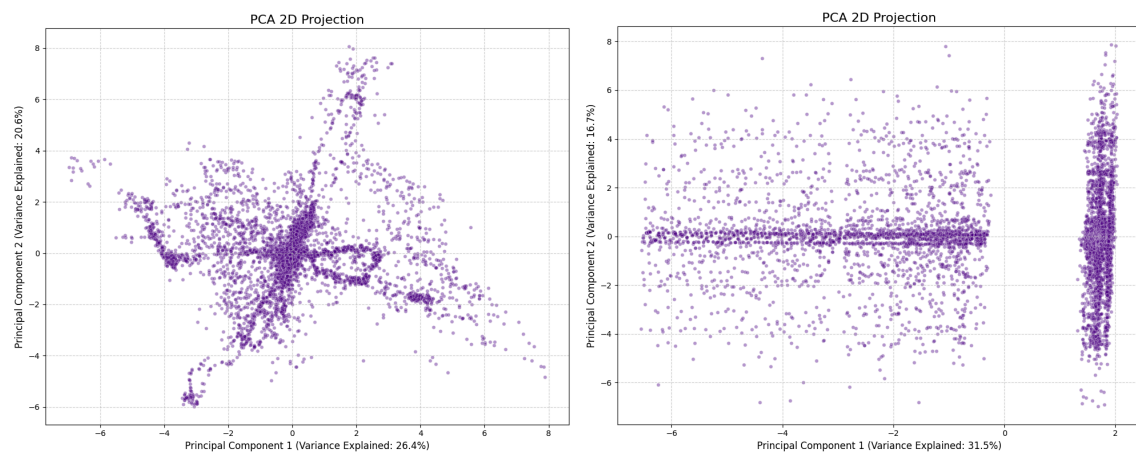


19개 피처의 선형적인 관계를 나타내는 히트맵을 보면, **Current J0**와 **Current J4**가 -0.758로 강한 역 상관관계를 보인다. **Speed J3**와 **Speed J5** 또한 강한 역 상관관계를 보인다.

모든 관절의 온도는 서로 0.99~1 사이의 매우 강한 상관관계를 보인다.



피쳐 분포 시각화 그래프를 보면 전류는 6개 관절 전류가 각각 다른 분포 형태를 가지고 각 변수가 서로 다른 정보를 담고 있을 가능성이 높다. 속도는 6개 관절 전부 0에 몰려있으니 변별력이 제한적이다. 온도는 상관계수가 매우 높았던 것처럼 사실상 온도 하나의 변수를 6번 측정한 것과 같다.



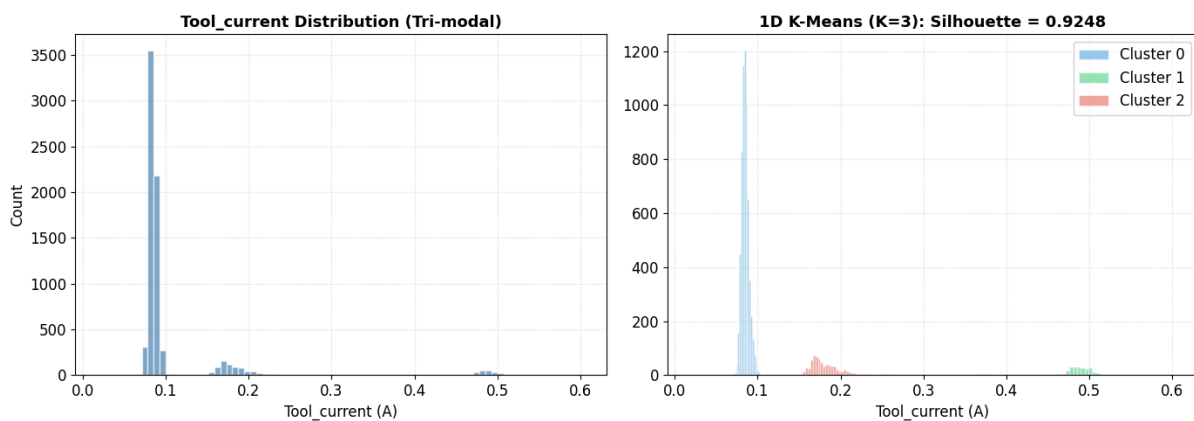
첫번째는 전류와 속도만으로 PCA를 한 것이고 두번째는 온도를 추가한 PCA 투영 산점도 그래프이다.

첫번째 그래프에서 전류·속도 12개를 PCA로 2D 투영한 결과, 중앙에서 여러 방향으로 뻗어나가는 방사형 구조가 관찰되었다. 이는 데이터에 복수의 운영 상태 패턴이 혼재함을 시사하며, 클러스터링을 통해 이를 분리할 수 있는 근거가 된다.

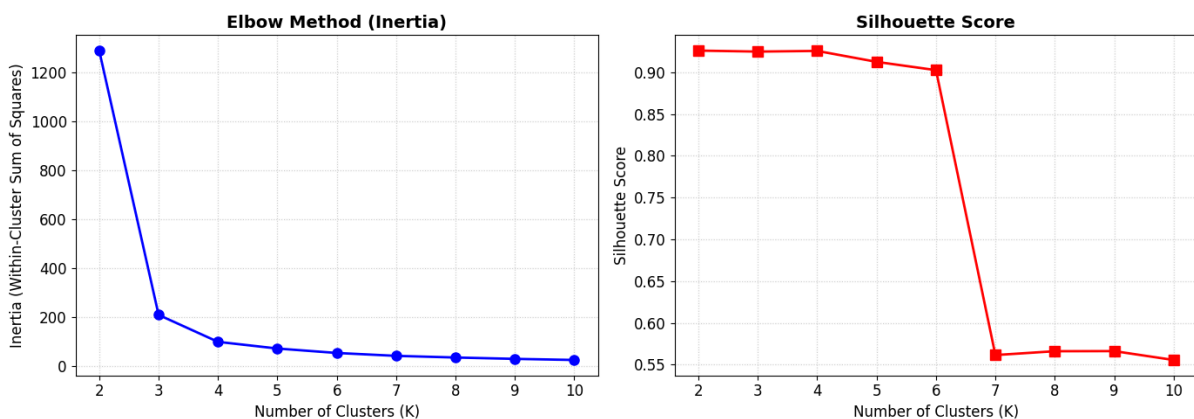
두번째 그래프에서 추가로 온도 6개 변수를 포함하면 상관계수 0.99 이상의 중복 정보가 PC1을 지배하여 운영 상태 구조가 시각화에서 사라진다. 이는 온도 변수의 다중공선성이 분석에 미치는 부정적 영향을 보여주는 사례이기도 하다.

따라서, 원본 피쳐로 클러스터링을 진행 한 후, 클러스터 구조를 잘 보여주는 전류와 속도 PCA 그래프로 모델별 결과의 시각화를 진행한다.

2.5. 기존 주제의 문제점

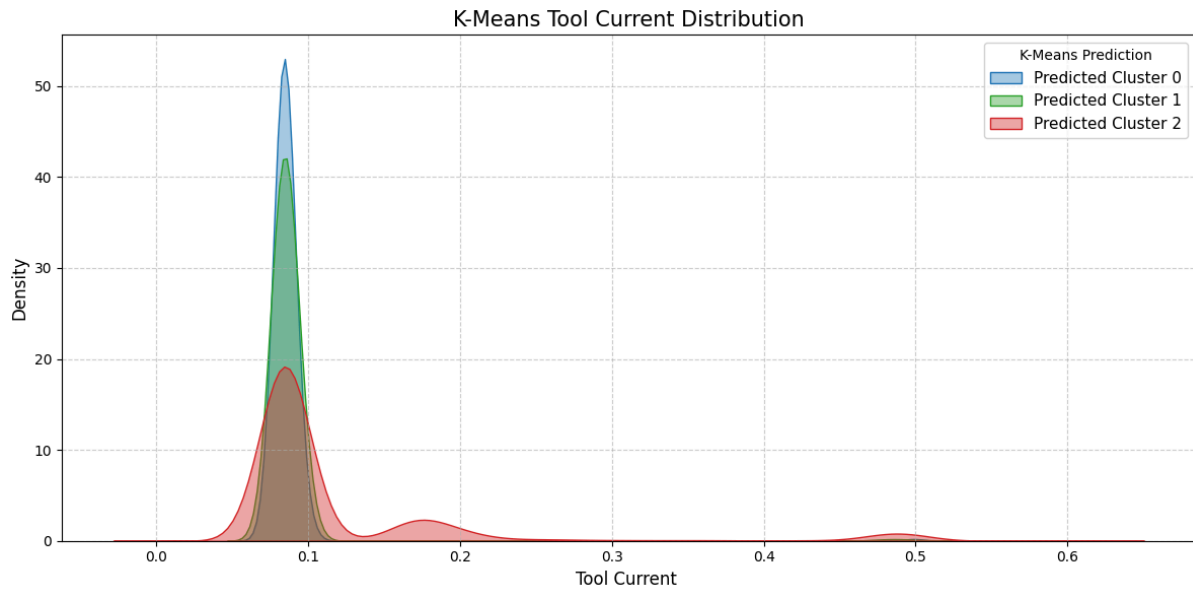


가장 처음에 선택한 주제는 Tool Current를 비롯한 다양한 변수가 데이터셋이 수집될때의 물건 하중을 잘 군집화할 수 있는지 확인하는것이였다.



이 주제에는 큰 문제점들이 있었다. 첫번째는 Tool current = 하중의 정답이라는 가정 자체의 문제점인데 따른 논리는 데이터셋을 모을때 1kg, 2kg, 3kg로 실험했으니 Tool Current를

K-means로 나누니 3개의 봉우리가 나왔다고 이 봉우리 3개가 곧 1kg, 2kg, 3kg라고
생각했다. Tool current 1차원으로 K=3 클러스터링은 Silhouette 0.92가 나왔으나 애초에
삼봉분포이므로 너무 당연한 결과다.



Tool Current를 제외한 피처를 통해 클러스터링을 진행했으나, 정답을 Tool Current와
비교했다. 즉, 피처가 군집을 어떻게 나누었는지는 신경쓰지 않고 무조건 하중과 Tool
Current에 매몰되어 결과를 내었다.

데이터에 하중같은 라벨이 어디에도 없고 Tool Current의 결과를 보고 우리가 추정한것이지
확인된 사실이 아니다. 데이터셋이 모을때 하중 말고도 속도와 그리퍼의 힘을 다르게
실험하였다. 즉, 하중의 무게보다 Tool Current는 그리퍼의 힘을 대변하고 있을 가능성이 더
높았다.

따라서, 다양한 피처들로 로봇의 운영 상태를 클러스터링을 통해 발견하는 주제로
변경하였다.

3. K-Means

```

df = pd.read_csv("https://raw.githubusercontent.com/kzming2007/UR3_Cobot_ML/refs/heads/main/dataset/ur3_cobotops.csv")
df_clean = df.dropna().copy()

df_clean['ProtectiveStop_bool'] = df_clean['Robot_ProtectiveStop'].apply(
    lambda x: True if str(x).strip().lower() == 'true' else False
)

# 피쳐 그룹
current_cols = ['Current_J0', 'Current_J1', 'Current_J2',
                'Current_J3', 'Current_J4', 'Current_J5', 'Tool_current']
speed_cols = ['Speed_J0', 'Speed_J1', 'Speed_J2',
              'Speed_J3', 'Speed_J4', 'Speed_J5']
temp_cols = ['Temperature_T0', 'Temperature_J1', 'Temperature_J2',
             'Temperature_J3', 'Temperature_J4', 'Temperature_J5']

group_A = current_cols # 7개
group_B = current_cols + speed_cols # 13개
group_C = current_cols + speed_cols + temp_cols # 19개

groups = {
    'Group A (Current, 7)': group_A,
    'Group B (Current+Speed, 13)': group_B,
    'Group C (All, 19)': group_C
}

print(f"데이터: {len(df_clean)}행 x {len(df_clean.columns)}열")
for name, cols in groups.items():
    print(f" {name}: {cols}")

```

✓ 0.3s

19개의 센서 측정값으로 정의되는 로봇의 운영 상태는 몇 가지로 구분되는가를 확인하기 위해 단순히 모든 변수를 한 번에 넣고 돌리는 것을 지양하고, 로봇의 물리적 상태를 단계적으로 확장해 가며 어떤 정보가 군집화에 가장 핵심적인지 파악하기 위해 피쳐 그룹을 3단계로 설계했다.

Group A는 전기적 하중 상태 (**Current**, 7개 변수)로 로봇이 현재 얼마나 높은 부하를 견디고 있는가를 파악하기 위해 각 관절의 전류와 그리퍼 전류로 구성했다.

Group B는 동적 운전 상태 (**Current + Speed**, 13개 변수)로 로봇이 움직이고 있는지, 정지해있는지의 동적 맥락을 추가하기 위해 속도 변수를 추가로 구성했다.

Group C는 종합 운영 상태로 19개 변수를 모두 사용했다. 가동 시간에 따른 열적 변화를 포함한 전체 센서 정보를 평가하기 위함이다.


```
scalers = {}
X_scaled = {}

for name, cols in groups.items():
    scaler = StandardScaler()
    X_scaled[name] = scaler.fit_transform(df_clean[cols].values)
    scalers[name] = scaler
    print(f"{name}: {X_scaled[name].shape}")

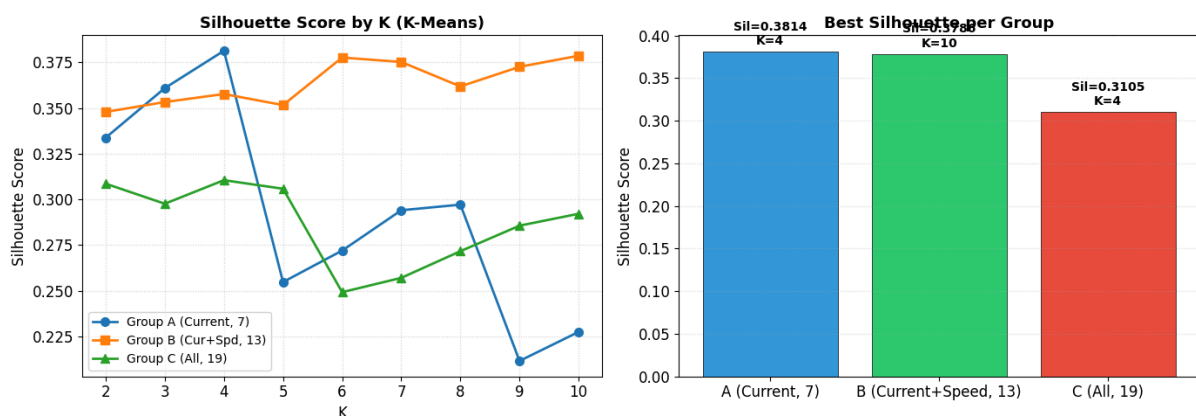
print("\n✅ StandardScaler 적용 완료")
```

0.0s

Group A (Current, 7): (7355, 7)
Group B (Current+Speed, 13): (7355, 13)
Group C (All, 19): (7355, 19)

✅ StandardScaler 적용 완료

모든 그룹의 변수들에 Standard Scaling을 적용했다.

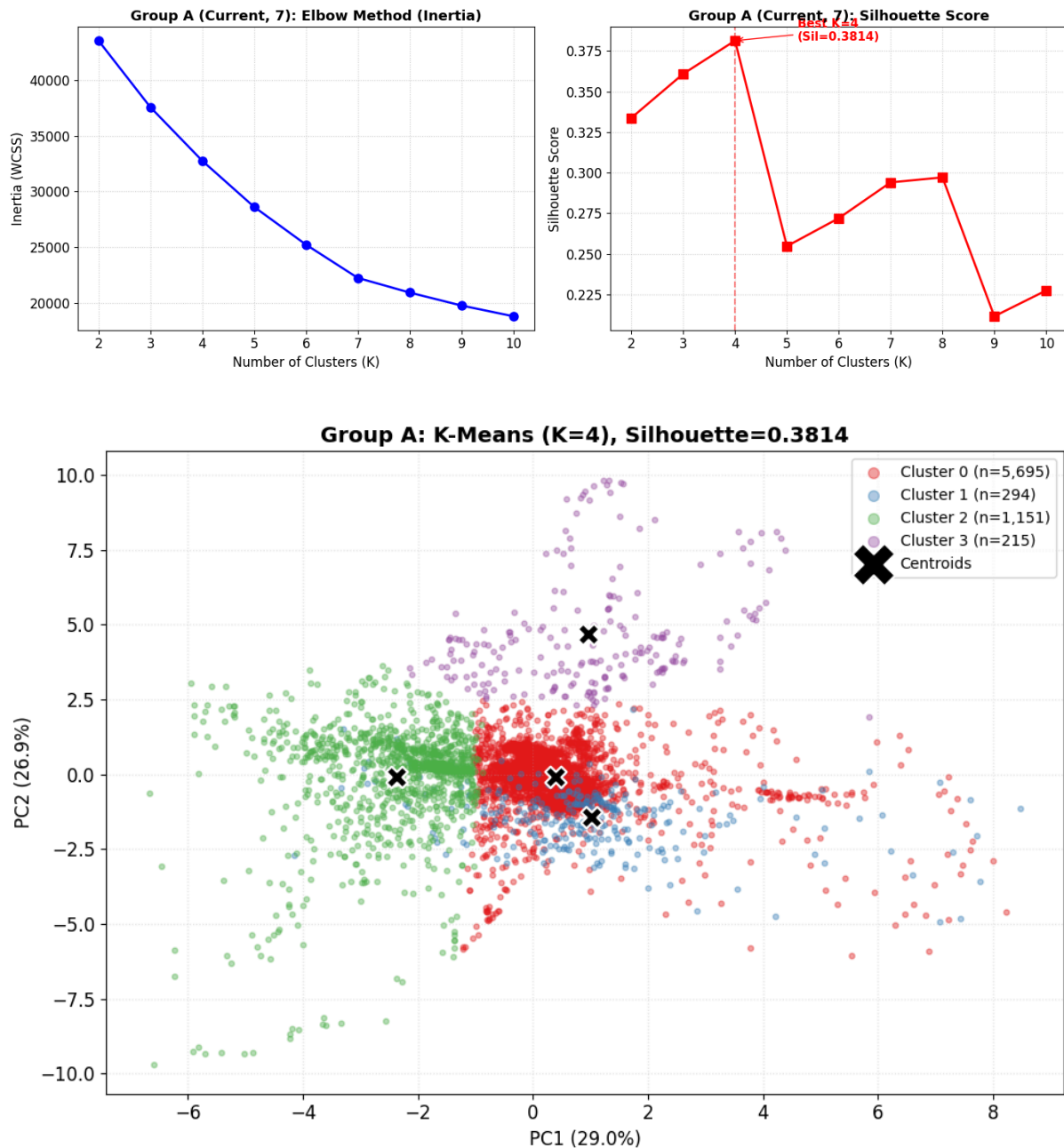


Group A는 k가 4일때 실루엣 0.3814을 형성하고 5부터 점수가 하락한다. 비록 절대적인 실루엣 스코어가 높은건 아니지만 데이터가 4개의 상태로 자연스럽게 나뉘를 의미한다. 전류 변수만으로도 로봇의 운영 상태를 구분하기에 가장 적합한 조건으로 해석할 수 있다.

Group B는 실루엣 점수 자체는 A와 크게 차이가 없지만 특정 k에서 뾰족한 피크를 만들지 못하고 k는 2부터 10까지 점수가 평탄하게 요동친다. 실루엣 스코어가 가장 높은 k인 10이 자동으로 선택되었으며 과적합 위험이 높다. 뒤이어 소개할 Group C도 k가 4에서 최적을 보였음으로 비교의 공정성을 위해 Group B도 k를 4로 고정하여 진행한다.

Group C는 가장 낮은 실루엣 점수인 0.3105를 기록했다. 서로 상관계수가 0.99인 온도 변수 6개가 추가되면서 k-means가 거리를 계산할때 불필요한 노이즈로 작용했다고 예상할 수 있다.

아래는 그룹 A의 결과다.



전체 데이터의 약 77%를 차지하는 Cluster 0이 중앙에 밀집되어있다.

나머지 다른 상태의 데이터들이 각기 다른축을 향해 뻗어가는 형태를 보이고있다.



이 히트맵은 피쳐들의 군집별 평균값을 보여주며, 각 군집이 어떤 상태인지를 구체적으로 설명한다.

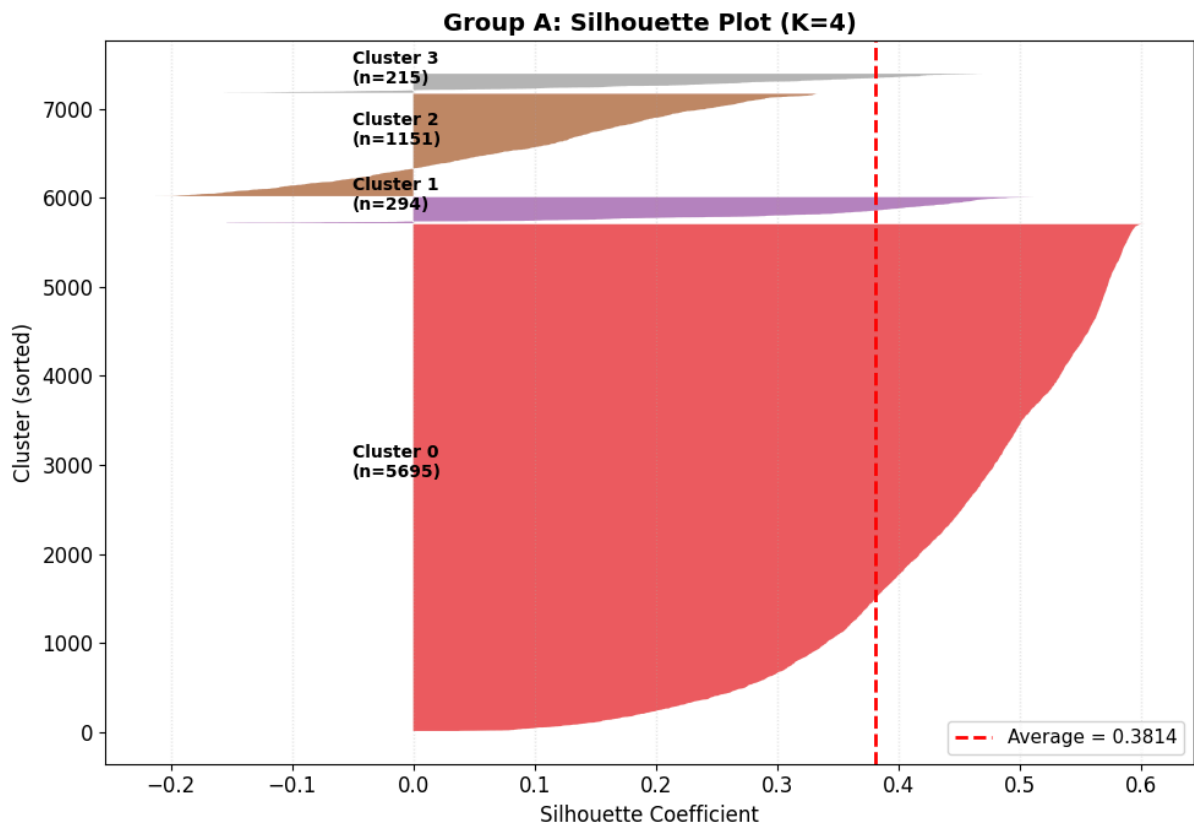
Cluster 0은 77%로 모든 관절의 전류값과 **tool current**가 0 근처 또는 평균적인 수준이다.

로봇이 무리한 부하 없이 가장 일반적이고 정상적인 궤적으로 작업을 수행 중인 가동 상태를 대변한다고 보인다.

Cluster 1은 전체 데이터의 약 4%를 차지하며, 다른 관절 전류는 비교적 평균적인 수준인 반면 **Tool Current**가 0.459로 가장 높게 나타난다. 이는 로봇팔 자체의 운동보다는 그리퍼 말단에서 추가적인 부하가 발생하는 상태를 나타낼 가능성이 있다. 따라서 그리퍼의 파지 조건이나 작업 대상의 특성에 의해 **Tool Current**가 증가한 운영 상태로 해석할 수 있다.

Cluster 2는 전체 데이터의 약 15.6%를 차지하며, **J0** 전류는 상대적으로 높고 **J1**과 **J2** 전류는 큰 음수 방향 값을 보인다. 이는 특정 자세 또는 궤적에서 여러 관절에 비대칭적인 부하가 집중되는 상태를 시사한다. 따라서 로봇팔이 작업 공간 내 특정 방향으로 크게 움직이거나 관절 모터에 비교적 높은 부하가 가해지는 운영 상태로 추정된다.

클러스터 3은 2.9%로 J0과 J1,J2가 강하게 역회전하고 동시에 J4 관절의 전류가 급격히 상승하는 독특한 패턴을 보인다. 이는 로봇의 몸통을 크게 틀면서 손목을 꺾는 등 특수하고 제한적인 궤적을 수행하는 상태로 추정된다.



이 그래프는 개별 데이터 포인트들이 자신이 속한 군집에 얼마나 잘 맞는지, 혹은 다른 군집에 잘못 할당되었을 가능성이 있는지를 보여준다. 붉은 점선은 전체 평균 실루엣 점수를 나타낸다.

클러스터 0은 전에도 확인했듯이 전체 데이터의 대다수를 차지하며 음수값을 가지는 데이터는 없다. 또한 대부분의 데이터가 전체 평균선을 크게 상회하는 점수를 기록하고있다.

즉, 정상 상태의 패턴이 매우 뚜렷하게 나타남을 보여준다.

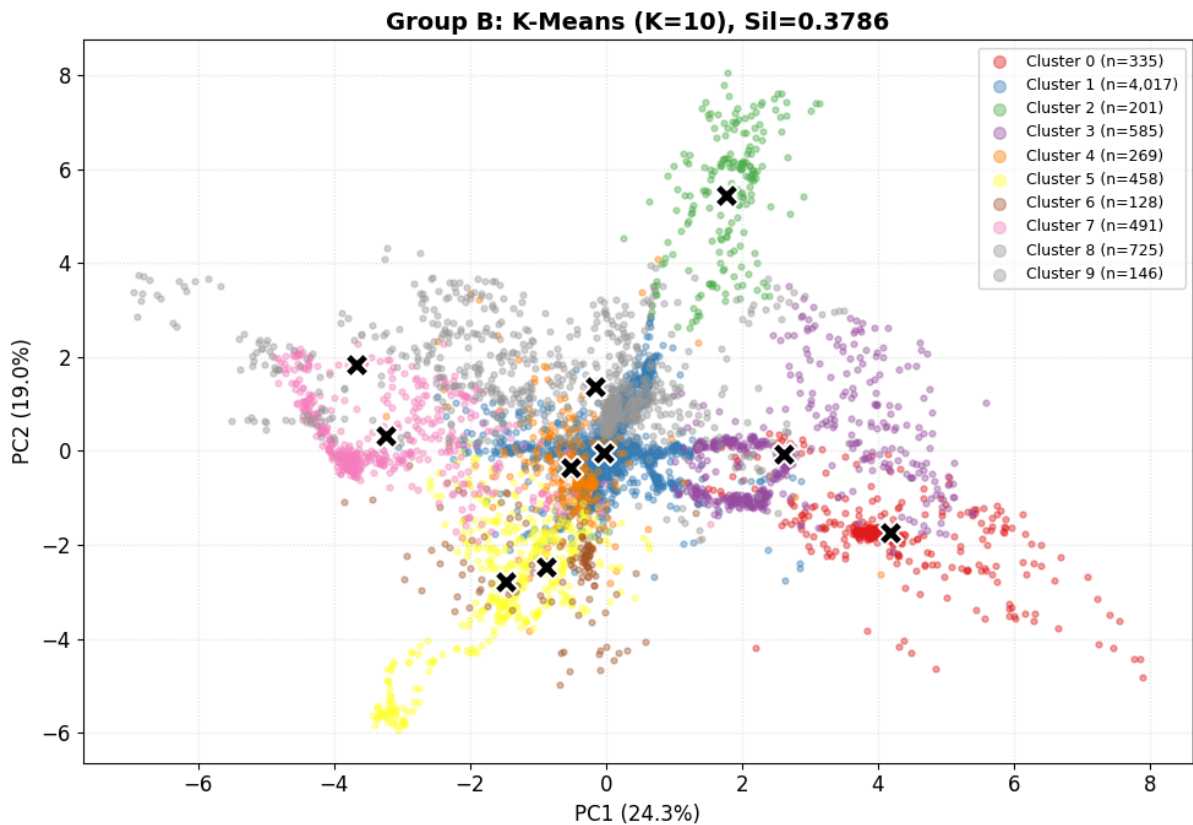
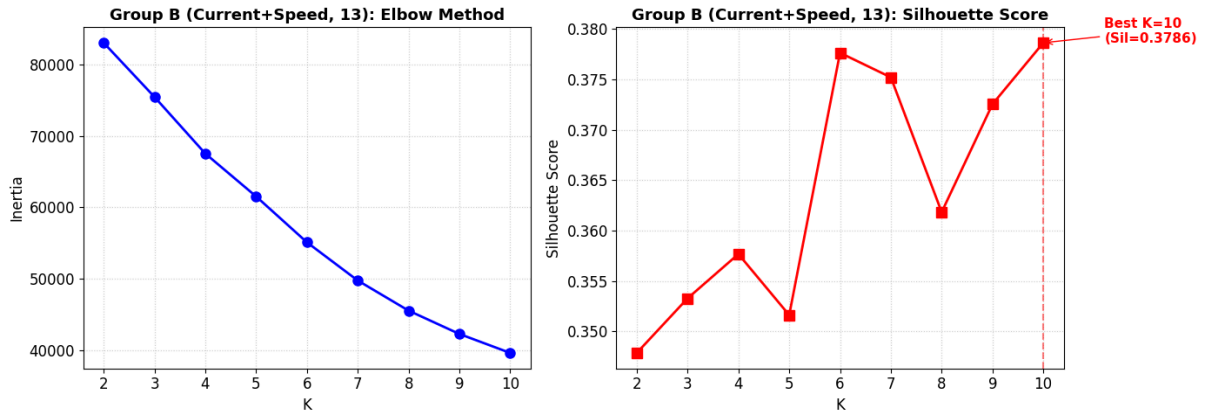
클러스터 2는 데이터는 두번째로 많지만 모양이 좋지 못하다. 그래프 왼쪽으로 뻗죽하게 튀어나온 음수 영역이 존재한다. 또한 전체 평균선에 미치지 못하는 데이터의 비율이 상당히 높다.

즉, 클러스터 2로 넘어가는 경계가 명확하지 않고 다소 모호함을 나타낸다.

이는 특정 관절 부하가 점진적으로 증가하면서 정상 상태와 특수 상태 사이의 경계 영역에 위치한 데이터들이 다수 존재함을 시사한다.

클러스터 1과 3은 데이터 수는 적지만 평균선을 상회하는 데이터들이 다수 존재하며, 음수 비율은 상대적으로 적다. 비록 발생 빈도는 낮지만, 클러스터 1이나 클러스터 3은 그 자체로 상당히 독특한 패턴을 가지고 있어 다른 상태들과 비교적 잘 구분되고 있음을 보여준다.

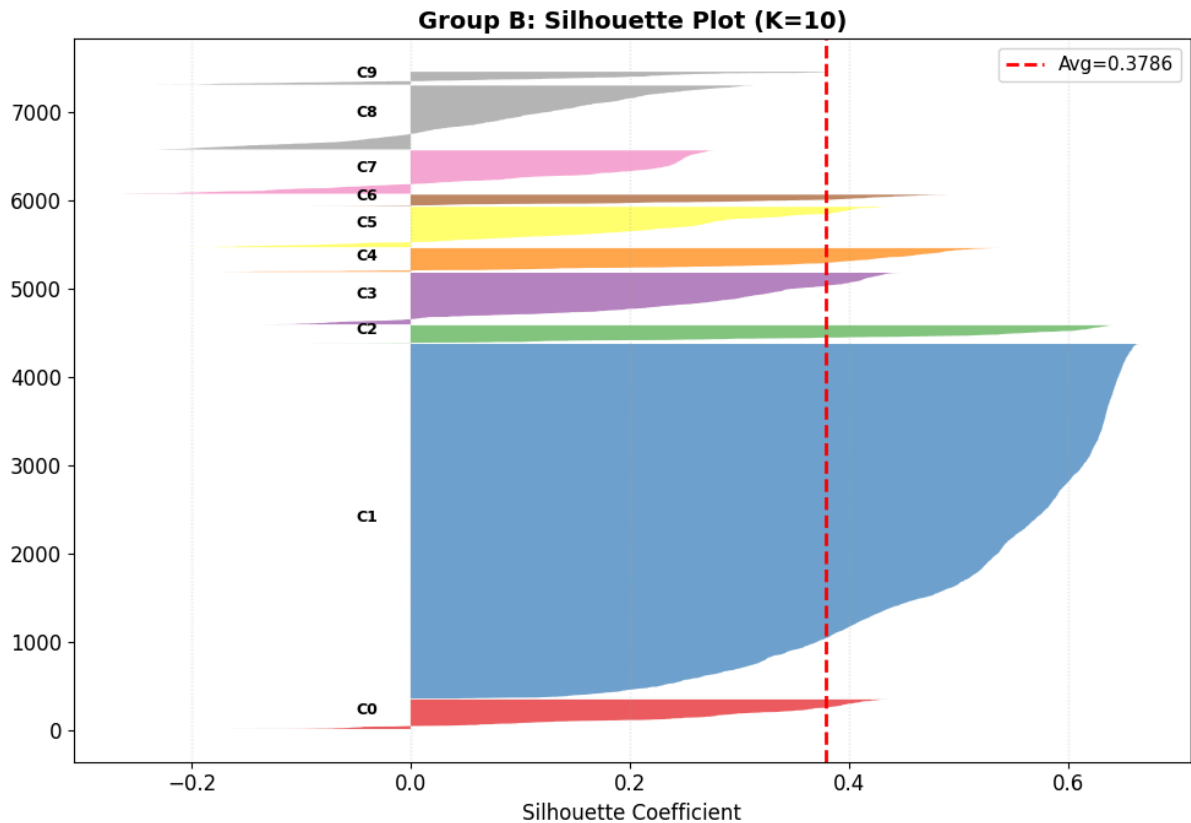
아래는 그룹 B의 결과다.



전류와 속도를 사용한 그룹 B의 엘보우 그래프는 뚜렷하게 꺾이는 구간 없이 완만하게 하락한다. 실루엣 그래프는 특정 K값에서 피크를 형성하지 못하고 톱니바퀴처럼 크게 요동치며, K=10에서 최고점을 기록했다. 이는 데이터 내에 자연스럽게 명확한 군집의 수가 존재하지 않음을 의미한다.

k=10으로 군집화된 결과를 2차원 PCA로 투영해본 결과, 10개의 군집이 마치 조각보처럼 잘게 쪼개져있으며, 특히 데이터가 가장 밀집된 중앙영역조차 여러개의 군집으로 무의미하게 분할되어 경계가 심하게 겹쳐있다.

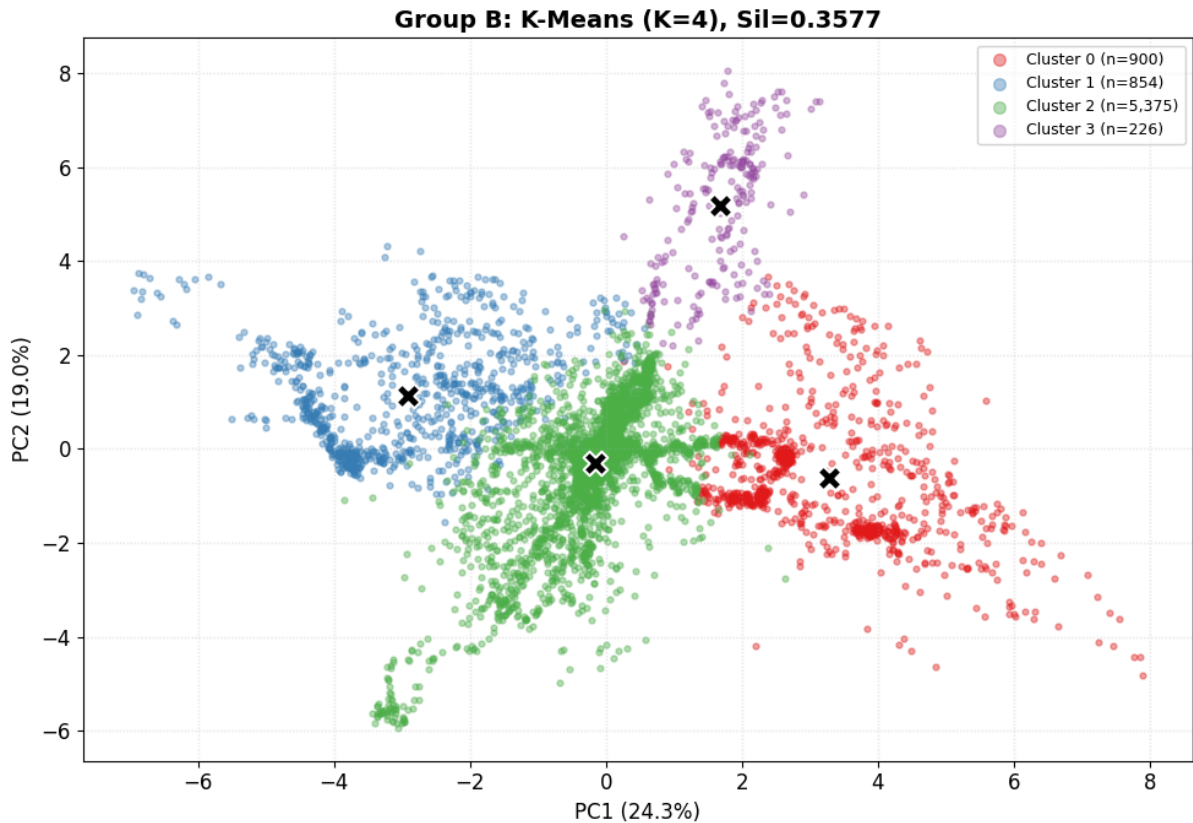
이는 지나친 세분화로 인해 알고리즘이 기계적으로 가장 높은 값인 10을 선택했지만 10개로 나눈 군집은 해석이 불가능하다. 속도 변수들이 0 근처에 몰려있어 군집을 무의미하게 잘게 쪼갠 것으로 보인다.



k가 10인 실루엣 플롯을 보면, 클러스터 1군집 하나가 전체 데이터의 대부분을 차지하고 나머지 9개의 군집은 데이터 수가 매우 적고 얇은 띠 형태로 나타난다.

클러스터 1과 2정도를 제외한 거의 모든 소규모 군집에서 왼쪽으로 길게 뻗은 음수 영역이 관찰된다.

비지도 학습에서는 실루엣 점수가 가장 높은 K가 항상 가장 해석 가능한 군집 구조를 의미하는 것은 아니다. Group B의 경우 K=10에서 가장 높은 실루엣 점수를 보였으나, 군집이 지나치게 세분화되어 물리적 운영 상태를 설명하기 어려웠다. 따라서 해석 가능성과 모델 간 비교의 일관성을 고려하여 K=4에서도 추가적인 군집 분석을 수행하였다.



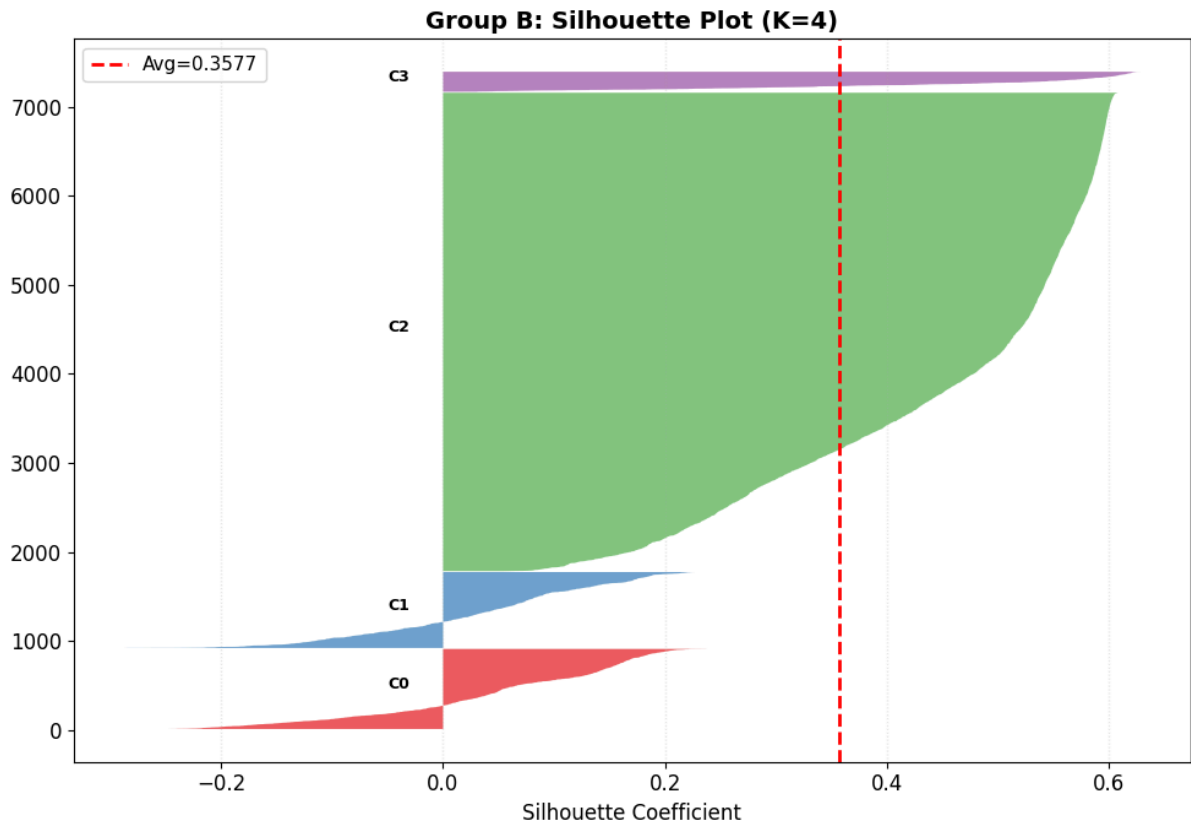
클러스터를 4개로 나눈 결과 어지러운 형태가 사라지고, 중앙의 클러스터 2를 중심으로 3개의 뿔어나가는 가지 형태가 나타났다.

그룹 A에서 보았던 4가지 물리적 상태 구조를 되찾은 것으로, 속도 변수가 추가된 상태에서도 본질적인 4개의 운영 상태가 존재함을 확인할 수 있다.

실루엣 점수는 0.3577정도로 A보다 낮게 측정되었다.

전반적으로 Group B의 PCA 산점도에서도 4개의 운영 상태가 관찰되지만, 실루엣 스코어는 Group A보다 낮게 나타났다. 다만 PCA 설명력은 Group A가 55.9%, Group B가 43.3%로 나타나, Group A의 2차원 투영 결과가 원본 데이터의 분산을 상대적으로 더 많이 보존하였다. PCA는 군집 품질을 직접 평가하는 지표는 아니지만, 시각화 관점에서는 Group A의 군집 구조가 보다 명확하게 관찰되었다. 따라서 시각적 해석의 관점에서는 Group A의 군집 구조가 더 명확하게 나타난다고 볼 수 있다.

속도를 추가했음에도 실루엣 스코어와 PCA의 설명력이 떨어졌다는것은 속도 변수가 로봇의 운영 상태를 구분하는데 유의미한 정보를 제공하기 보다는 군집 구조를 명확히 구분하는 데 기여하지 못했을 가능성을 시사한다.



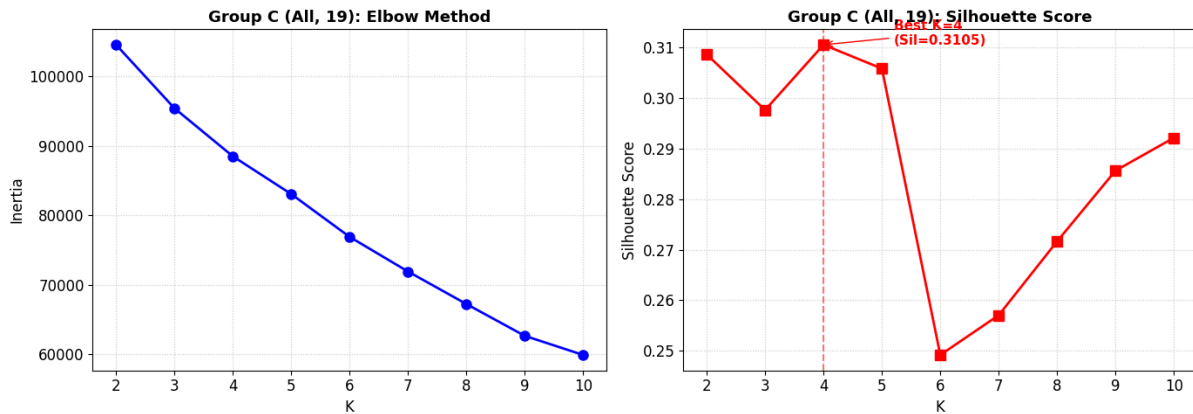
가장 정상 상태인 클러스터 2를 제외하고, 클러스터 0과 1의 군집 형태가 심하게 일그러져있다.

두 군집 모두 평균선을 넘는 데이터가 하나도 없으며 절반에 가까운 데이터가 깊은 음수 영역으로 뻗어있다.

이는 클러스터 0과 1로 분류된 수많은 데이터들이 자신이 속한 군집보다 거대한 클러스터 2 군집에 수학적으로 더 가깝다는 것을 의미한다. 앞서, 전류 피처만 사용한 그룹 A 상태일때는 단 하나의 군집에서만 일부 음수가 발생했던 것과 대비된다.

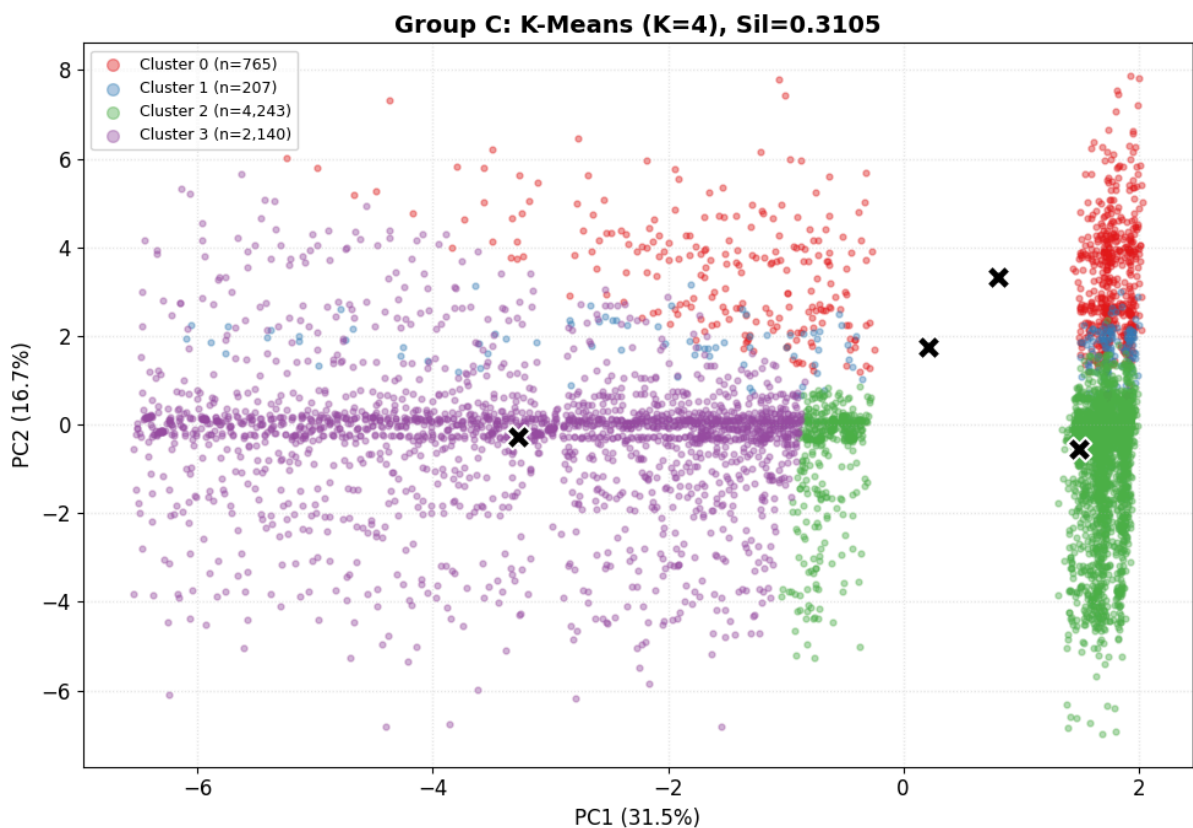
대부분 0에 머물러 있는 속도 변수가 군집 구조를 구분하는 데 제한적인 정보를 제공하기에 데이터간의 거리가 왜곡되고 군집 간의 경계가 붕괴된것으로 추측된다.

아래는 그룹 C의 결과다.



마찬가지로 k는 4에서 최고점을 찍었으나 그 점수가 세 그룹중 가장 낮다.

새로 추가된 온도 변수가 군집 형성에 도움을 주기보다는 군집 품질을 저하시켰을 가능성이 있다.

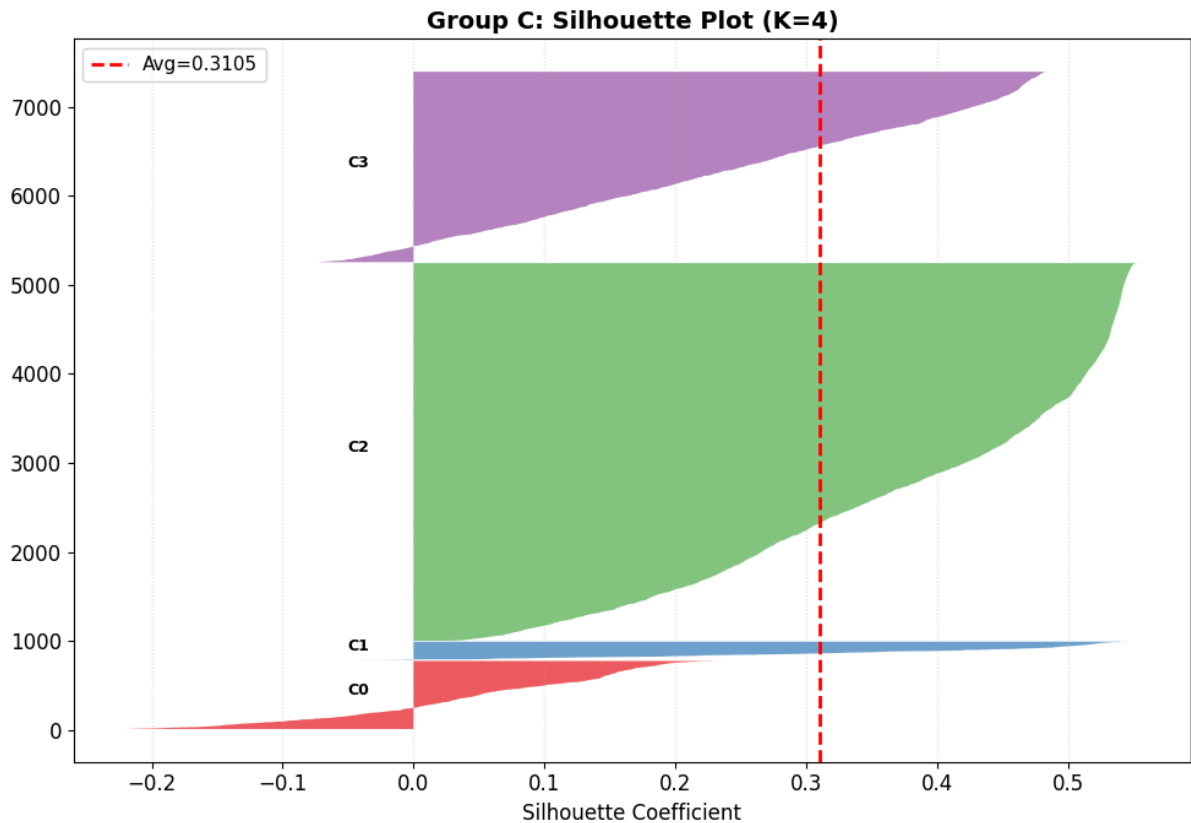


온도 변수를 추가하니, Classification에서 보았던 두개의 거대한 기동 형태로 나뉘었다.

각 Centroids를 보면, 왼쪽은 클러스터 3가 전체를 차지하고, 오른쪽은 클러스터 2가 기동에 있고 1과 0은 위아래로 무의미하게 잘린듯 나뉘어있다.

원인을 분석하자면 온도 변수 6개는 서로 0.99 이상의 완벽한 상관관계를 가진다. 즉, 1개의 정보가 6번이나 중복되어 알고리즘에 투입된 상태라고 할 수 있다.

이 중복 정보가 거리 계산을 지배해 그래프의 x축이 사실상 온도를 대변하게 되면서 온도 상태로 분류해 버린 것으로 보인다.

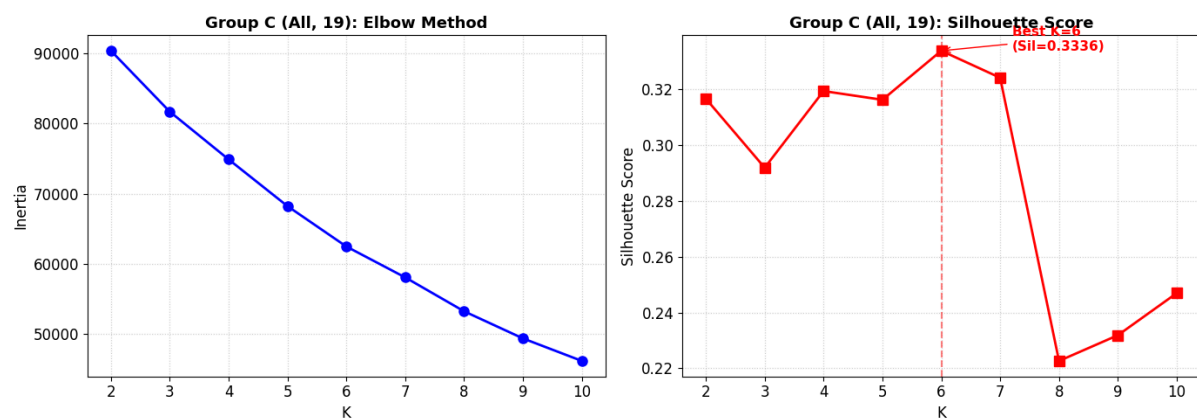


실루엣 스코어를 보면, 클러스터 2와 3이 가장 많은 데이터를 가지고 있지만, 3에서 음수 부분이 보인다. 또한 클러스터 0은 평균선을 넘지 못하고 깊은 음수 영역으로 뻗어있다.

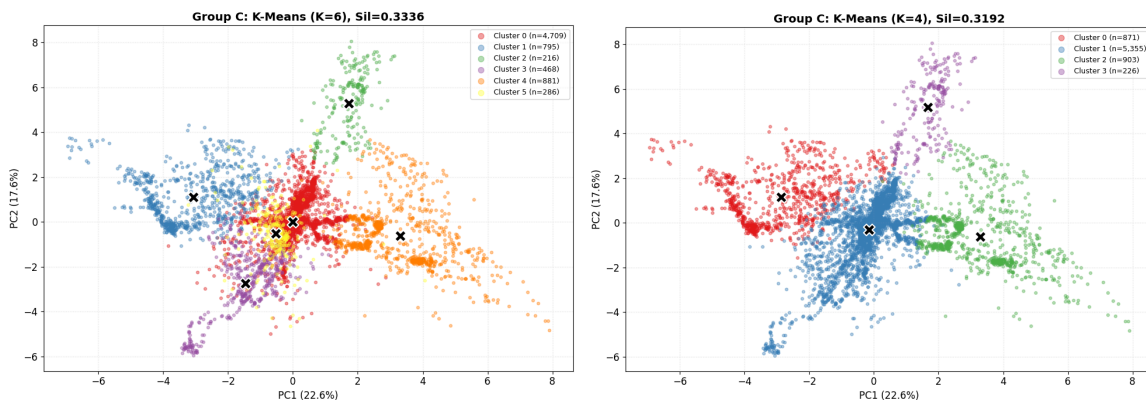
```
# 피쳐 그룹
current_cols = ['Current_J0', 'Current_J1', 'Current_J2',
               'Current_J3', 'Current_J4', 'Current_J5', 'Tool_current']
speed_cols   = ['Speed_J0', 'Speed_J1', 'Speed_J2',
               'Speed_J3', 'Speed_J4', 'Speed_J5']
temp_cols    = ['Temperature_T0', 'Temperature_J1', 'Temperature_J2',
               'Temperature_J3', 'Temperature_J4', 'Temperature_J5']
df_clean['Temperature_Mean'] = df_clean[temp_cols].mean(axis=1)

group_A = current_cols # 7개
group_B = current_cols + speed_cols # 13개
#group_C = current_cols + speed_cols + temp_cols # 19개
group_C = current_cols + speed_cols + ['Temperature_Mean'] # 14개
```

같은 정보가 6번 들어갔으므로, 만약 관절별 온도를 평균낸 단 하나의 온도값만 들어가면 어떻게 되는지 관찰하기 위해 추가적으로 실험해보았다.

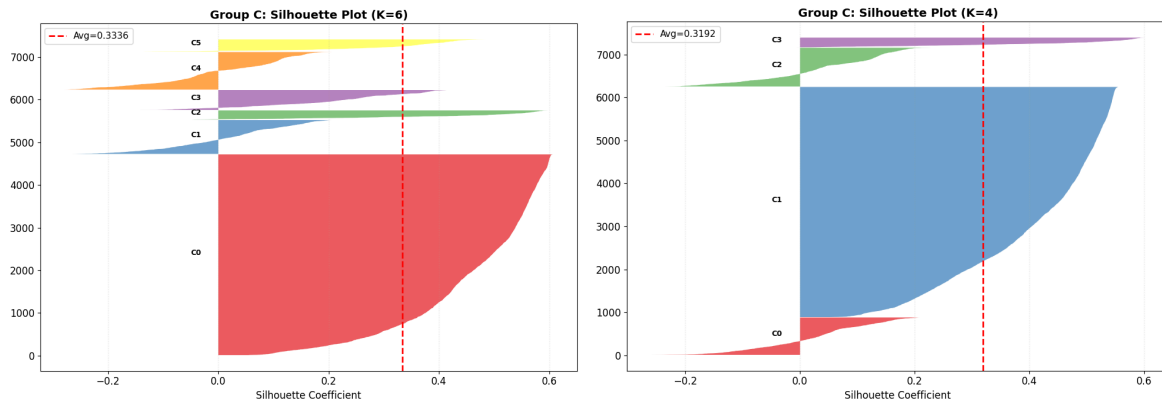


가장 높은 실루엣 점수는 k가 6일때의 0.3336으로 나왔다. 이는 앞선 그룹 A, B와 비교하면 여전히 낮다.



가장 높은 K가 6일때와 4일때의 PCA 그래프를 보면 이전처럼 온도로 인해 데이터가 좌우 두 기둥으로 완전히 갈라지는 현상은 사라졌다.

데이터 차원의 왜곡을 방어했고 모델이 온도에 집착하지 않고 다시 물리적 가동 궤적을 바라보기 시작했다고 추측할 수 있다.



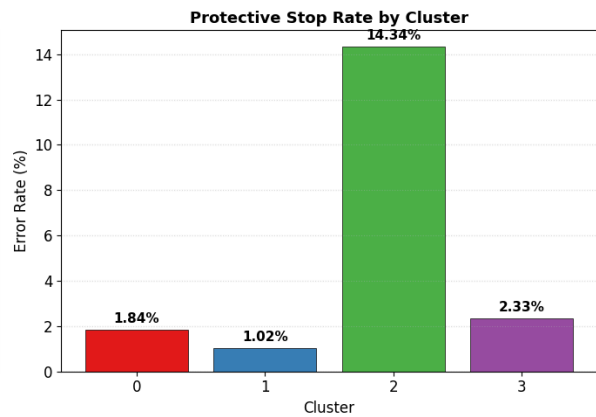
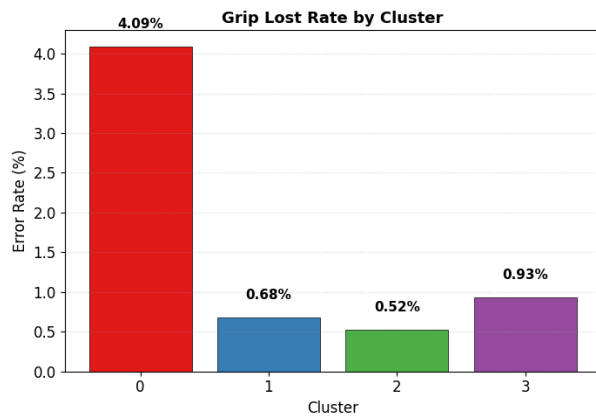
실루엣 플롯을 비교해보면 k가 6일때는 중심 군집을 제외한 다른 군집들의 데이터 절반 이상이 깊은 음수 영역으로 빠져있다.

데이터의 거시적 구조는 되찾았으나, 세밀한 군집의 경계는 붕괴됨을 알 수 있다.

온도는 전류나 속도처럼 상태에 따라 즉각적으로 변하는 값이 아니라, 로봇이 켜져있는 동안 서서히 오르고 내리는 연속적인 시계열 성향이 강하다.

따라서 평균을 내어 다중공산성을 제거했음에도 불구하고, 특정 운영상태를 나누는데 부적합한 노이즈임을 시사한다.

결론적으로 **Group A**는 세 그룹 중 가장 높은 실루엣 점수와 가장 해석 가능한 군집 구조를 보였으며, 로봇의 운영 상태를 구분하는 데 가장 적합한 피처 구성으로 판단된다.



[교차표] Cluster vs Error Type				
error_type	GripLost	Normal	ProtectiveStop	Total
KM_Cluster_A				
0	233	5360	102	5695
1	2	289	3	294
2	6	980	165	1151
3	2	208	5	215
Total	243	6837	275	7355

가장 높은 성능을 보였던 **Group A**를 에러 라벨을 사용해 외부 검증을 진행하고 **K-Means**의 결과를 총 정리한다.

Cluster 0 (77.4%)는 전체 데이터의 대부분을 차지하며, 모든 관절의 전류가 평균적인 범위에 분포한다. 따라서 로봇이 일반적인 작업을 수행하는 대표적인 정상 운전 상태로 해석할 수 있다.

Cluster 1 (4%)은 다른 관절 전류는 비교적 평균적인 수준인 반면 **Tool Current**가 가장 높게 나타난다. 따라서 그리퍼 말단에 상대적으로 높은 부하가 발생하는 운영 상태로 해석할 수 있다.

Cluster 2 (15.7%)는 J1과 J2 전류가 큰 음수 방향 값을 보이며, 다른 군집과 구별되는 전류 패턴을 나타낸다. 이는 특정 자세 또는 궤적에서 관절 모터에 상대적으로 높은 부하가 집중되는 운영 상태로 추정된다.

Cluster 3 (2.9%)는 J0 전류가 음수 방향으로 크게 나타나고 J4 전류가 급격히 증가하는 독특한 패턴을 보인다. 따라서 일반적인 작업 상태와는 구별되는 특수한 자세 또는 제한적인 궤적을 수행하는 운영 상태로 해석할 수 있다.

에러 라벨을 사용해 외부 검증을 진행한 결과, **Grip lost**가 발생할 확률은 평범한 정상 가동 상태인 **Cluster 0**에서 가장 높은 발생률인 **4.09%**를 보였다. 전체 243건 중 233건이 여기서 발생했다.

이는 로봇이 그립을 놓치는 현상이 모터의 과부하나 특이한 비틀림 자세 때문에 발생하는것이 아님을 시사한다.

무리하지 않은 일상적인 이동중에 미끄러짐, 외부 충돌 등 관절 전류 이외의 외부적 요인으로 인해 발생할 확률이 높다는 중요한 인사이트를 얻었다.

Protective Stop은 275건 중 약 60%인 165건이 **Cluster 2**에서 발생하였다. 또한 **Cluster 2**의 에러 발생률은 **14.34%**로, 다른 군집에 비해 약 7배 이상 높게 나타났다.

앞선 히트맵 분석에서 **Cluster 2**는 J1과 J2 관절 전류가 크게 나타나는 특징을 보였다.

이러한 결과는 특정 관절에 높은 부하가 집중되는 운영 상태에서 **Protective Stop**이 상대적으로 자주 발생할 가능성을 시사한다. 따라서 **Cluster 2**는 로봇의 보호 정지와 밀접한 관련이 있는 운영 상태로 해석할 수 있다.

로봇 관제 시스템에서 실시간으로 관절 전류를 모니터링 하다가 데이터가 **Cluster 2**의 패턴에 진입하는 순간 알람을 울리거나 궤적 속도를 낮추면, **Protective Stop** 에러의 약 60%를 사전 예방할 수 있음을 시사한다.

4. DBSCAN

최적의 피쳐 조합으로 판명된 **Group A** 데이터를 활용하여, 밀도 기반 군집화 알고리즘인 **DBSCAN**을 적용한다.

K-Means 모델은 모든 데이터를 특정 군집에 강제로 할당하기 때문에, 경계면의 데이터들이 군집의 질을 저하시킬 위험이 존재한다.

이러한 한계를 보완하기 위해, 군집의 개수를 알고리즘이 스스로 결정하고 형태가 불규칙한 군집을 찾아내는 데 유리한 **DBSCAN**을 도입하였다.

이를 통해 로봇의 연속적인 동작 궤적 속에서 높은 밀도로 응집된 핵심 상태들을 추출하고, 군집에 포함되지 않는 노이즈를 격리하여 데이터의 비선형적인 구조를 분석하고자 한다.

```
eps_range = np.arange(0.5, 3.1, 0.25)
min_samples_range = [5, 7, 10, 14]

results = []
for eps in eps_range:
    for ms in min_samples_range:
        db = DBSCAN(eps=eps, min_samples=ms)
        labels = db.fit_predict(X)

        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        n_noise = (labels == -1).sum()
        noise_pct = n_noise / len(labels) * 100

        sil = -1
        if n_clusters >= 2 and n_clusters < len(labels) - n_noise:
            mask = labels != -1
            if len(set(labels[mask])) >= 2:
                sil = silhouette_score(X[mask], labels[mask])

        results.append({
            'eps': round(eps, 2), 'min_samples': ms,
            'n_clusters': n_clusters, 'noise_count': n_noise,
            'noise_pct': round(noise_pct, 1), 'silhouette': round(sil, 4)
        })

df_results = pd.DataFrame(results)

# 유효한 결과만 필터 (군집 2개 이상, 노이즈 50% 미만)
df_valid = df_results[(df_results['n_clusters'] >= 2) &
                      (df_results['noise_pct'] < 50) &
                      (df_results['silhouette'] > 0)].copy()

print(f"[그리드 탐색 결과] 전체 {len(df_results)}개 조합 중 유효: {len(df_valid)}개")
print(f"\n{'='*75}")
print(df_valid.sort_values('silhouette', ascending=False).head(15).to_string(index=False))

# 최적 파라미터
if len(df_valid) > 0:
    best = df_valid.loc[df_valid['silhouette'].idxmax()]
    print(f"\n 🏆 최적 파라미터: eps={best['eps']}, min_samples={int(best['min_samples'])}")
    print(f"   → 군집 수={int(best['n_clusters'])}, 노이즈={best['noise_pct']}%, Silhouette={best['silhouette']}")
```


DBSCAN은 K-Means와 달리 군집의 개수를 사전에 지정하지 않는 대신, 탐색 반경(**eps**)과 최소 데이터 개수(**min samples**) 파라미터에 매우 민감하게 반응한다. 따라서 **Grid Search**를 통해 로봇 운영 데이터에 가장 적합한 밀도 조건을 도출하였다.

군집의 수인 **n_clusters**는 하나의 거대한 군집만 생성되거나 모두 노이즈로 처리되는 과소적합 결과를 배제하기 위해 **2개** 이상으로 설정했다.

노이즈 비율인 **noise_pct**는 DBSCAN이 군집에 속하지 못한다고 판단한 노이즈의 비율을 제한함으로 모델이 정상 궤적마저 노이즈로 오판하는것을 방지하기 위해 **50%** 미만으로 설정한다.

노이즈를 제외하고 오직 형성된 핵심 군집들이 얼마나 단단하게 응집되었는지만 평가하여 최적의 파라미터를 선정하도록 설계했다.

[그리드 탐색 결과] 전체 44개 조합 중 유효: 35개

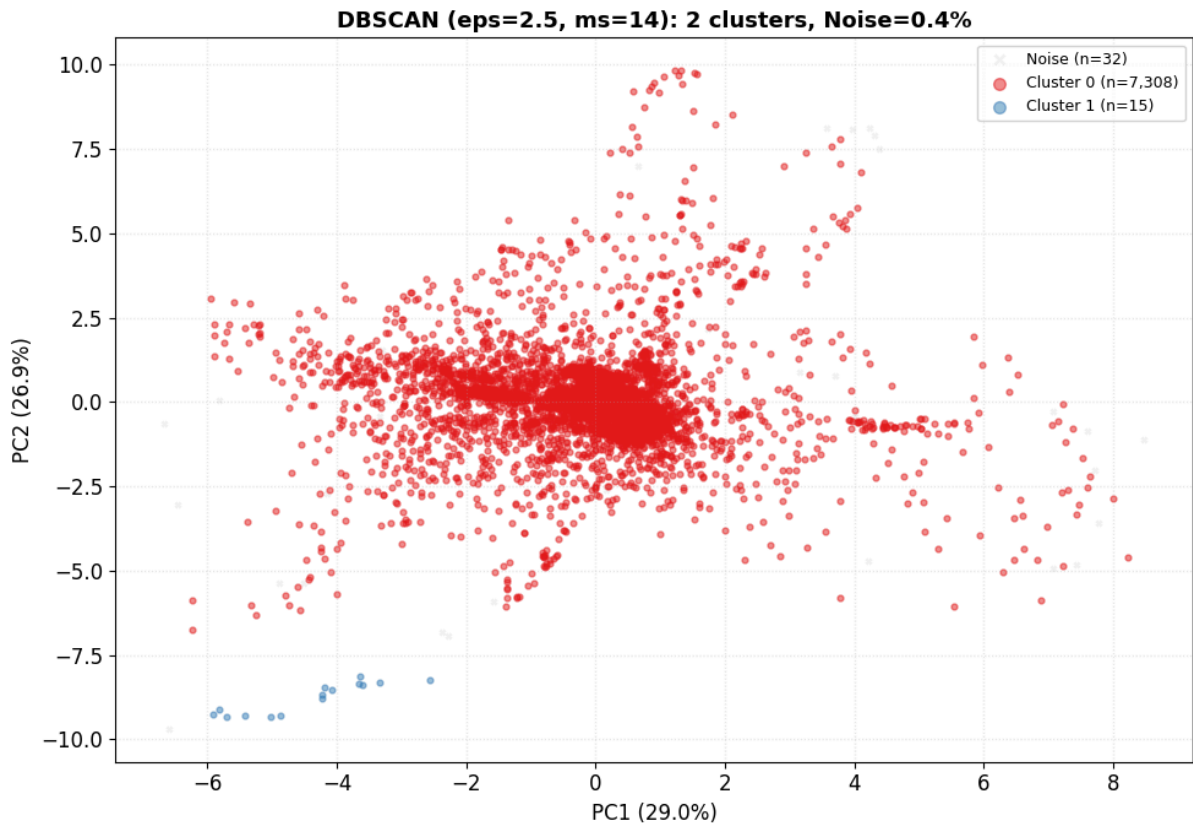
eps	min_samples	n_clusters	noise_count	noise_pct	silhouette
3.00	14	2	4	0.1	0.6984
2.50	14	2	32	0.4	0.6984
2.75	14	2	18	0.2	0.6964
2.00	7	2	57	0.8	0.6938
2.25	7	2	22	0.3	0.6856
2.00	14	3	151	2.1	0.6465
1.75	14	3	199	2.7	0.6449
2.25	14	3	86	1.2	0.6220
2.25	5	2	10	0.1	0.5713
1.50	10	6	208	2.8	0.5623
2.25	10	3	44	0.6	0.5596
2.00	5	3	30	0.4	0.5552
2.00	10	4	79	1.1	0.5338
1.75	7	6	91	1.2	0.5071
1.75	10	7	129	1.8	0.4329

🏆 최적 파라미터: **eps=2.5, min_samples=14**
→ 군집 수=2, 노이즈=0.4%, Silhouette=0.6984

grid search 결과 모델들을 보면 실루엣 스코어가 전반적으로 K-Means보다 훨씬 높지만, 이는 좋은 수치가 아니다.

실루엣이 0.6984인 가장 높은 모델을 보면, K-Means와 상반되게 군집을 2개로 설정했다.

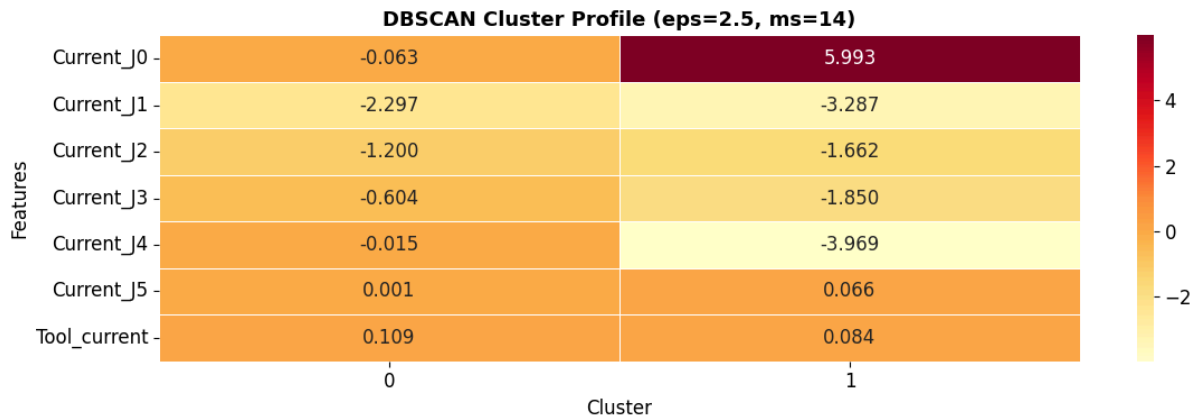
이것은 DBSCAN이 거대한 덩어리 하나와 나머지 극소수의 이상치 군집으로 나눈것으로 사실상 과소적합되어 군집화를 하지 않은것과 같다고 할 수 있다.



PCA 시각화를 해석해보면 Cluster 0은 99.4%를 차지하며 전체 데이터가 하나의 덩어리다

Cluster 1은 0.2% 정도로 가장 멀리 떨어진 극단적인 이상치다.

K-Means는 같은 데이터에서 4가지 운영 상태를 발견했지만, DBSCAN은 이 데이터의 밀도 관점에서는 거의 하나의 연속적인 덩어리라고 판단했다고 여겨진다.



실루엣 스코어가 가장 높았던 **eps=2.5, min_samples=14** 조건의 군집 히트맵을 확인한 결과이다.

군집 0은 일상적인 궤적을 나타내고, 군집 1은 J0 관절 전류가 평균 대비 5.993으로 폭증하는 극단적인 예러 상태 단 하나만을 포착했다. 이는 K-Means 모델이 팔을 뻗는 등의 다양한 부하 상태들을 전혀 구분하지 못하고, 오직 평범한 상태와 관절이 터지기 직전의 극단적 튀는 상태 단 2분법으로만 세상을 바라보고 있음을 의미한다.

```
[K-Means ↔ DBSCAN 라벨 일치도]
ARI = 0.0337
NMI = 0.0273

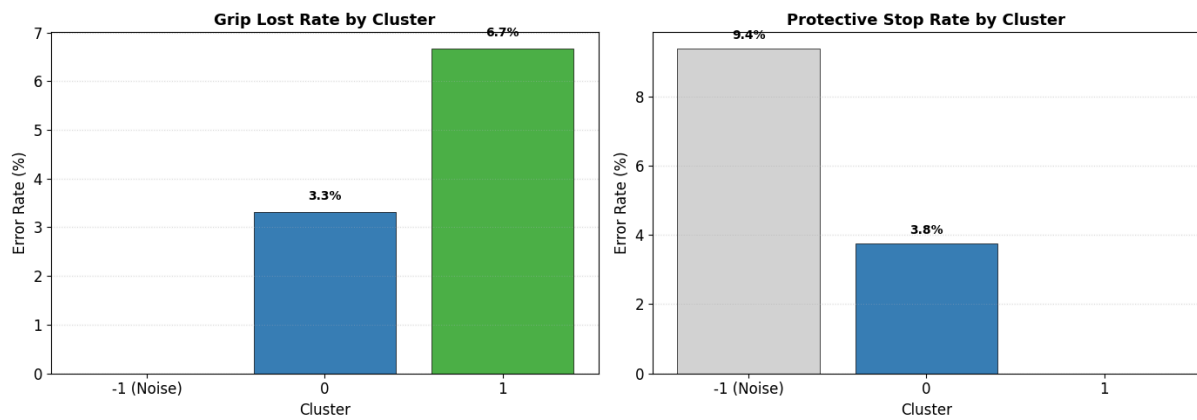
[교차표] K-Means Cluster vs DBSCAN Cluster
```

DBSCAN \ KMeans	-1	0	1	Total
0	2	5693	0	5695
1	15	279	0	294
2	9	1127	15	1151
3	6	209	0	215
Total	32	7308	15	7355

K-Means와 DBSCAN의 교차표를 보면, K-Means Cluster 1(그리퍼 하중, 294개)는 DBSCAN의 노이즈 15개로 DBSCAN이 밀도가 낮다고 판단한 노이즈가 K-Means의 그리퍼 하중 상태와 겹친다.

K-Means Cluster 2(과부하, 1151개)는 DBSCAN Cluster 1로 분리된 15개로 가장 극단적인 과부하 데이터 15개만 DBSCAN이 별도 군집으로 인식했다.

ARI=0.0337, NMI=0.0273 라는 것은 두 알고리즘의 라벨 일치도가 거의 0에 가까워 완전히 다른 관점에서 데이터를 바라보고 있다는 의미이다.



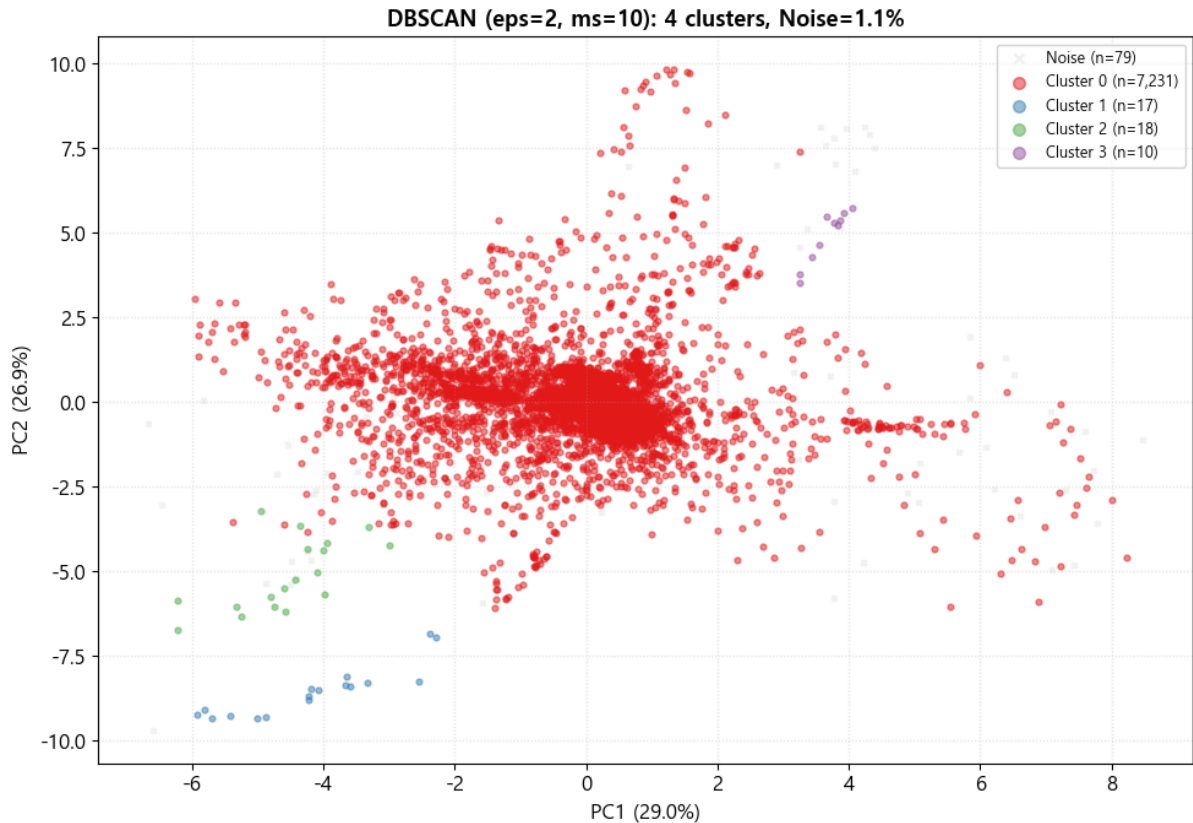
에러 라벨을 통한 외부 검증을 진행한다.

노이즈로 간주된 Cluster -1에서 Protective Stop 9.38%이 나왔고 Cluster 1에서 Grip lost가 6.7%가 나왔다.

DBSCAN이 어느 군집에도 속하기 어렵다고 판단한 이상 데이터가 에러와 연결된다.

하지만 표본 수가 너무 적어 신뢰하기는 어렵다.

DBSCAN은 이 데이터를 밀도 관점에서 하나의 연속적 덩어리로 판단하여 K-Means와 같은 세밀한 4가지 상태 분류에 실패하였다. 이는 로봇의 운영 상태가 불연속적으로 끊어지는 것이 아니라 연속적인 스펙트럼 위에서 점진적으로 변화하는 데이터 구조이기 때문이다.



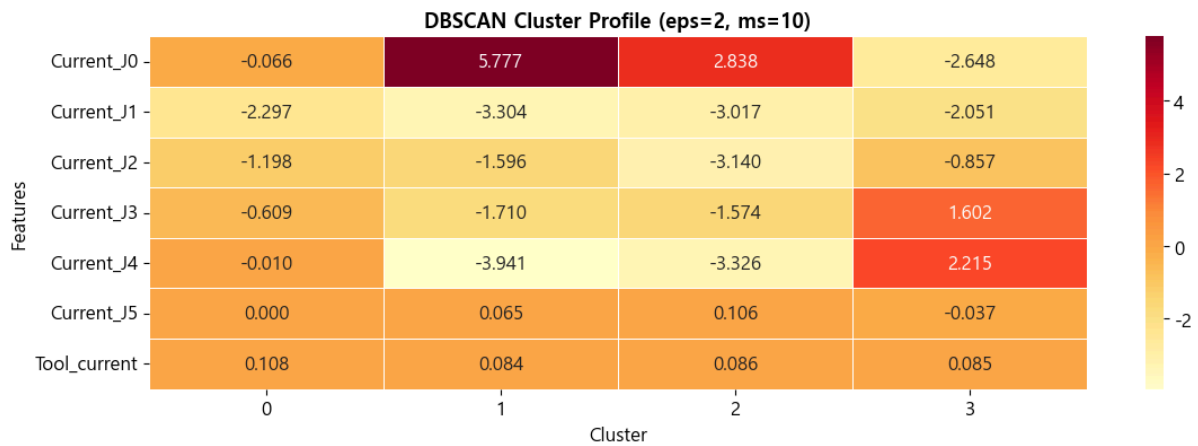
그리드 탐색 표에서 군집 수가 4개인 조합($\text{eps}=2.0$, $\text{min_samples}=10$, $\text{Sil}=0.5338$)을 선택하여 DBSCAN의 군집이 4개가 되도록 파라미터를 바꿔보았다.

PCA 시각화를 보면 여전히 Cluster 0이 중앙에 가장 큰 군집을 형성하고 가장 끝단 영역에 극소수의 점들만 별도의 군집으로 떨어져나왔다. DBSCAN은 이 데이터들이 정상적인 밀도 흐름에서 완전히 벗어난 이질적인 이상치로 잘라낸 것이다.

각 군집의 외곽을 따라 등성등성 퍼져있는 79개의 노이즈가 존재한다. 어느 군집의 기준 밀도에도 도달하지 못할만큼 매우 찰나의 순간에 발생했거나 센서의 일시적 튜트 현상으로 발생한 노이즈들을 걸러냈음을 보여준다.

K-Means가 거리를 기준으로 데이터 공간을 4개로 분할했던 것과 달리, DBSCAN은 데이터의 전체적인 흐름이 밀도적으로 단절 없이 이어져 있음을 보여주었다. 따라서 로봇의 전반적인 운영 상태를 거시적으로 분류하는 데에는 K-Means가 더 적합하다고 여겨지는 반면, DBSCAN은 주류 데이터의 흐름에서 완전히 동떨어진 극단적 이상치와 노이즈를

정밀하게 고립시키는 데 탁월한 성능을 보였으므로, 치명적 에러를 사전 감지하는 이상치 탐지 목적으로 활용 가치가 있는 것 같다.



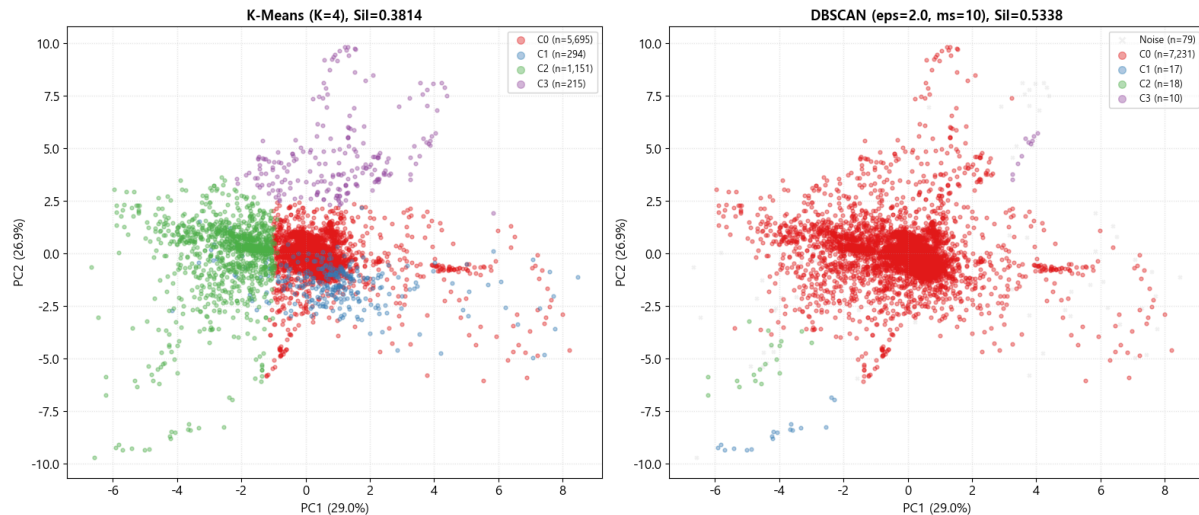
Cluster 0은 98.3%을 차지하며 모든 전류값이 0 근처이거나 전반적으로 평이한 수준을 보인다. **K-Means**는 이 안에서도 미세한 차이를 쪼개어 정상, 과부하, 그리퍼 이슈 등으로 나누었지만 **DBSCAN**은 이정도 변화량은 정상적인 작업 환경 내의 움직임으로 간주해 하나로 통합했음을 보여준다.

Cluster 1은 17개의 데이터로 구성되며, **J0** 전류가 5.777로 매우 높게 나타나는 반면 **J4**는 큰 음수 방향 값을 보인다. 이는 베이스 관절과 손목 관절에 서로 다른 방향의 부하가 집중되는 특이한 운영 상태를 나타내는 것으로 해석할 수 있다.

Cluster 2는 **J0**에 높은 양의 전류가 나타나는 동시에 **J1, J2, J4**는 음수 방향으로 큰 값을 보인다. 이는 여러 관절에 비대칭적인 부하가 동시에 발생하는 상태를 시사하며, 일반적인 작업 상태와 구별되는 특수한 전류 패턴으로 볼 수 있다.

Cluster 3은 **J0**에 역방향으로 부하가 걸리고 **J3, J4**는 정방향으로 부하가 걸려있는 상태이다. **Cluster 1**과 **Cluster 3**은 서로 다른 방향의 전류 패턴을 보이며, 다른 군집과 명확히 구분되는 특이 상태로 나타난다. 이는 **DBSCAN**이 정상 상태에서 벗어난 드문 운영 패턴을 효과적으로 분리해냈음을 보여준다.

Tool Current는 네 군집 모두 0.08~0.10 범위로 큰 차이를 보이지 않았다. 이는 K-Means가 Tool Current의 변화를 하나의 주요 군집 기준으로 활용한 것과 달리, DBSCAN은 여러 관절 전류의 복합적인 패턴을 더 중요하게 반영했음을 시사한다. 따라서 DBSCAN에서는 Tool Current의 영향력이 상대적으로 제한적으로 나타난 것으로 해석할 수 있다.



따라서, K-Means의 4개 군집은

Cluster 0 (77.4%): 정상 가동

Cluster 1 (4.0%): 그리퍼 하중

Cluster 2 (15.7%): 어깨/팔꿈치 과부하

Cluster 3 (2.9%): 특수 궤적

으로 로봇의 4가지 운영 상태를 보여주었다면

DBSCAN의 4개 군집은

Cluster 0 (98.3%, 7,231개): 사실상 모든 데이터

Cluster 1 (0.2%, 17개): 극단적 이상치 그룹 1

Cluster 2 (0.2%, 18개): 극단적 이상치 그룹 2

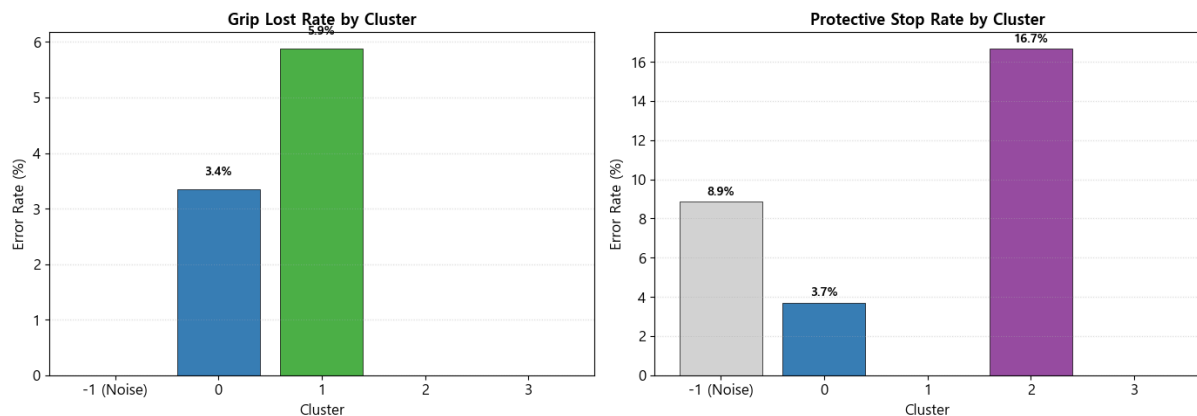
Cluster 3 (0.1%, 10개): 극단적 이상치 그룹 3

Noise (1.1%, 79개): 경계면 데이터

으로 거대한 정상과 극단적 이상치 3개 군집으로 나뉜다는것을 확인했다.

[교차표] K-Means Cluster vs DBSCAN Cluster						
DBSCAN	-1	0	1	2	3	Total
KMeans						
0	8	5687	0	0	0	5695
1	39	255	0	0	0	294
2	16	1100	17	18	0	1151
3	16	189	0	0	10	215
Total	79	7231	17	18	10	7355

K-Means가 정상 상태로 정의했던 Cluster 0은 DBSCAN에서도 99.8%가 Cluster 0으로 동일하게 분류되었다. 반면 K-Means가 서로 다른 운영 상태로 구분했던 Cluster 1, 2, 3의 데이터 대부분(약 1,544개)은 DBSCAN에서 하나의 거대한 Cluster 0으로 흡수되었다. 이는 K-Means가 전류 패턴의 상대적인 차이를 기준으로 세부적인 상태를 구분한 반면, DBSCAN은 데이터 밀도의 연속성을 더 중요하게 반영하기 때문으로 볼 수 있다. 따라서 K-Means에서 구분되었던 상태들이 밀도 관점에서는 명확히 분리된 집단이라기보다 서로 연결된 연속적인 영역으로 존재할 가능성을 시사한다.



Cluster 2 (18개)에서 Protective Stop 16.67%을 기록했다.

이 18개는 K-Means Cluster 2(과부하)에서 가장 극단적인 데이터이다.

DBSCAN은 이 극단치만 집어냈고, 에러 발생률이 16.67%로 K-Means의 Cluster 2(14.34%)보다 더 높다.

Noise (79개)에서 Protective Stop는 8.86%으로 DBSCAN만의 고유 기능인 노이즈 분리가 에러 연관 데이터를 잡아냈다.

DBSCAN은 동일한 Group A 피처에 대해 K-Means와는 근본적으로 다른 관점의 군집화를 수행하였다. 밀도 기반 알고리즘인 DBSCAN은 로봇의 운영 상태가 정상에서 과부하까지 밀도적으로 끊기지 않고 연속적으로 연결된 데이터 구조임을 드러냈으며, 전체 데이터의 98.3%를 하나의 군집으로 통합하고 나머지 극단적 이상치만을 분리하였다. 이는 K-Means가 거리 기반으로 공간을 강제 분할하여 도출한 4가지 운영 상태(정상/그리퍼 하중/과부하/특수 궤적)와 대조적이다. 그러나 DBSCAN이 분리한 극소수의 이상치 군집(Cluster 2)에서 Protective Stop 에러 발생률이 16.67%로 K-Means의 과부하 군집(14.34%)보다 더 높게 나타났다. 이는 DBSCAN이 비록 거시적인 상태 분류에는 부적합하지만, 가장 극단적인 위험 상태를 정밀하게 포착하는 이상치 탐지 도구로서는 K-Means보다 유용할 수 있음을 시사한다.

5. Hierarchical Clustering

앞서 K-Means는 거리를 기준으로 4개의 주요 운영 상태를 도출하였고, DBSCAN은 데이터가 밀도 관점에서 연속적으로 이어져 있음을 증명하였다. 세 번째 기법인 계층적 군집화를 적용하여 데이터의 숨겨진 구조를 다각도로 분석한다. 계층적 군집화는 데이터가 어떤 순서로 병합되는지를 덴드로그램이라는 트리 구조로 시각화할 수 있어, 로봇의 물리적 상태가 어떤 계층적 관계를 가지며 파생되는지 파악할 수 있을것으로 기대한다. 본 분석을 통해 K-Means가 도출한 4개의 군집 구조가 상향식 병합 과정에서도 동일하게 나타나는지 교차 검증하고, 최적의 군집화 전략을 종합적으로 도출하고자 한다.

```

df = pd.read_csv("https://raw.githubusercontent.com/kzming2007/UR3_Cobot_ML/refs/heads/main/dataset/ur3_cobotops.csv")
df_clean = df.dropna().copy()

df_clean['ProtectiveStop_bool'] = df_clean['Robot_ProtectiveStop'].apply(
    lambda x: True if str(x).strip().lower() == 'true' else False
)

# Group A (전류 7개) - K-Means에서 최적으로 선정된 피쳐 그룹
current_cols = ['Current_I0', 'Current_I1', 'Current_I2',
                'Current_I3', 'Current_I4', 'Current_I5', 'Tool_current']

scaler = StandardScaler()
X = scaler.fit_transform(df_clean[current_cols].values)

print(f"데이터: {X.shape[0]}행 {X.shape[1]}열 (Group A: 전류 7개)")

```

✓ 0.4s

데이터: 7355행 x 7열 (Group A: 전류 7개)

우선 데이터셋을 로드한 후 전류 7개를 묶고 Standard Scaling을 진행한다.

```

# 데이터가 7,355개로 많으므로 덴드로그램 시각화를 위해 샘플링
np.random.seed(42)
sample_idx = np.random.choice(len(X), size=min(2000, len(X)), replace=False)
X_sample = X[sample_idx]

linkage_methods = ['ward', 'complete', 'average']

fig, axes = plt.subplots(1, 3, figsize=(20, 6))

for idx, method in enumerate(linkage_methods):
    Z = linkage(X_sample, method=method)
    ax = axes[idx]
    dendrogram(Z, ax=ax, truncate_mode='level', p=5,
               no_labels=True, color_threshold=0,
               above_threshold_color='steelblue')
    ax.set_title(f'Dendrogram ({method.capitalize()} Linkage)',
               fontsize=13, fontweight='bold')
    ax.set_xlabel('Samples')
    ax.set_ylabel('Distance')

    # K=4 절단선 표시
    if method == 'ward':
        # Ward linkage에서 4개 군집이 되는 높이 추정
        max_d = Z[-3, 2] # 4번째로 큰 병합 거리 (n-k번째)
        ax.axhline(y=max_d, color='red', linestyle='--', alpha=0.7,
                  label=f'K=4 cut (d={max_d:.1f})')
        ax.legend(fontsize=9)

plt.tight_layout()
plt.show()

```

덴드로그램이란 X축은 개별 데이터를 나타내고, Y축은 두 데이터가 얼마나 다른지 거리를 나타낸다. 밑에서부터 자기와 가장 비슷한 데이터끼리 둘씩 짝을 지어 합친다.

계속 합치다보면 결국 꼭대기에서 1개의 거대한 덩어리가 된다.

∩ 모양의 세로선 길이가 길수록, 억지로 다른 성질의 것들을 합쳤다는 뜻이다. 반대로 밑에서 자잘하게 묶이는 것들은 서로 매우 비슷하다.

그룹과 그룹을 합칠 때 거리를 어떻게 잴것인지에 대한 3가지 **Linkage** 방식이 있다.

Ward 방식은 병합시 분산 증가량을 최소화한다. **K-Means**와 가장 유사한 철학이다.

Complete 방식은 두 군집 간 가장 먼 점 사이의 거리를 기준으로 한다.

Average 방식은 두 군집 간 모든 점 쌍의 평균 거리를 기준으로 한다.

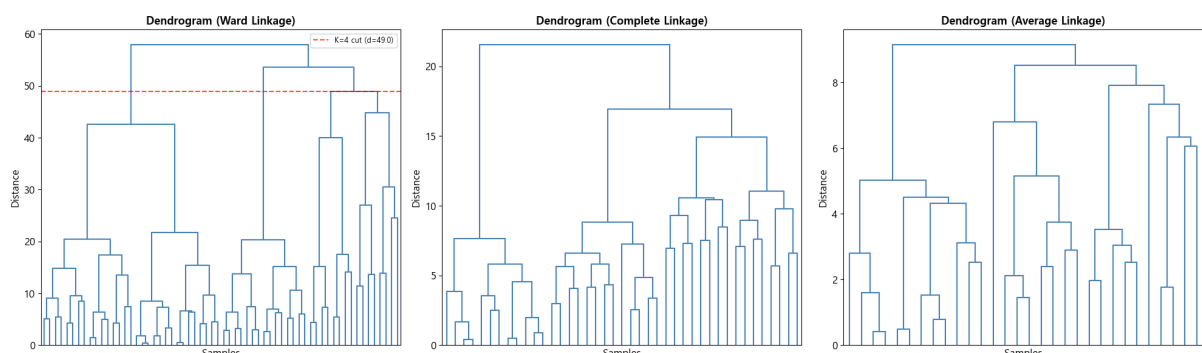
전체 데이터가 7355개이니 이걸 다 덴드로그램으로 그리면 밑바닥이 잘 보이지 않는다.

따라서 전체 데이터의 특징을 유지하면서 무작위로 2000개의 데이터만 뽑아서 그래프를 그린다.

앞서 다룬 3가지의 방식을 비교하기 위해 **ward**, **complete**, **average**를 리스트로 만들고 **for** 문을 사용해 **ward**, **complete**, **average**를 순서대로 3번 반복한다.

Z = linkage(X_sample, method=method) 를 사용해 서로 얼마나 거리가 먼지 계산해 족보를 만들고 계산된 족보를 바탕으로 실제 나무를 그린다.

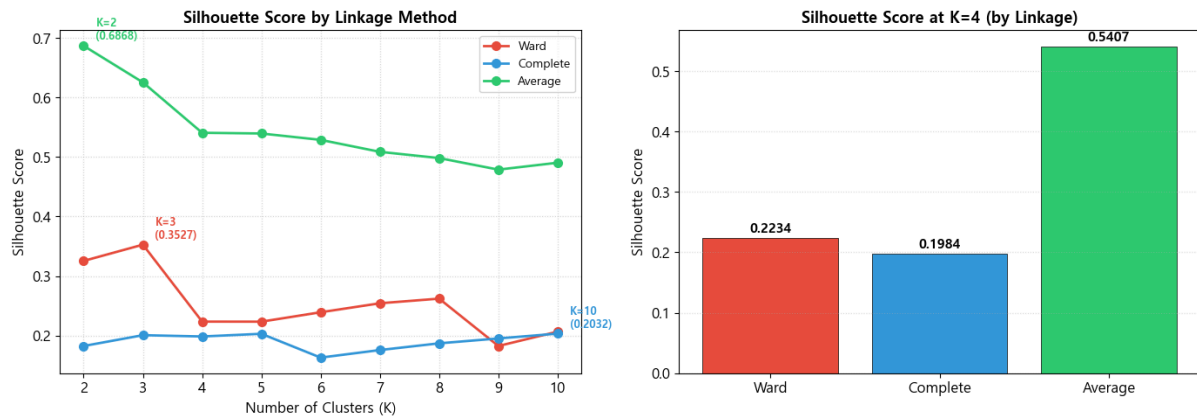
Ward 그래프를 그릴때, **K-Means**에서 운영 상태가 **k = 4**일때 가장 잘 나뉜다는 것을 확인했다. 그렇다면 같은 방식을 사용하는 **Ward** 덴드로그램 트리에도 거대한 4개의 묶은 가지가 자연스럽게 나타나는지 확인해보기 위해 4번째로 큰 병합 거리에 붉은 선을 그린다.



분석 결과 **Average**나 **Complete** 방식은 특정 거대 군집이 나머지 데이터를 순차적으로

흡수하는 현상이 발생하여 군집의 변별력이 떨어졌다.

반면, 군집내 분산을 최소화하는 **Ward** 방식은 그래프가 다른것보다 균형잡힌 계층 구조를 형성하였다. **Ward** 방법의 최상단 구조를 살펴보면 빨간 점선으로 거리를 절단했을때 4개의 기둥이 형성되는것을 볼 수 있다.



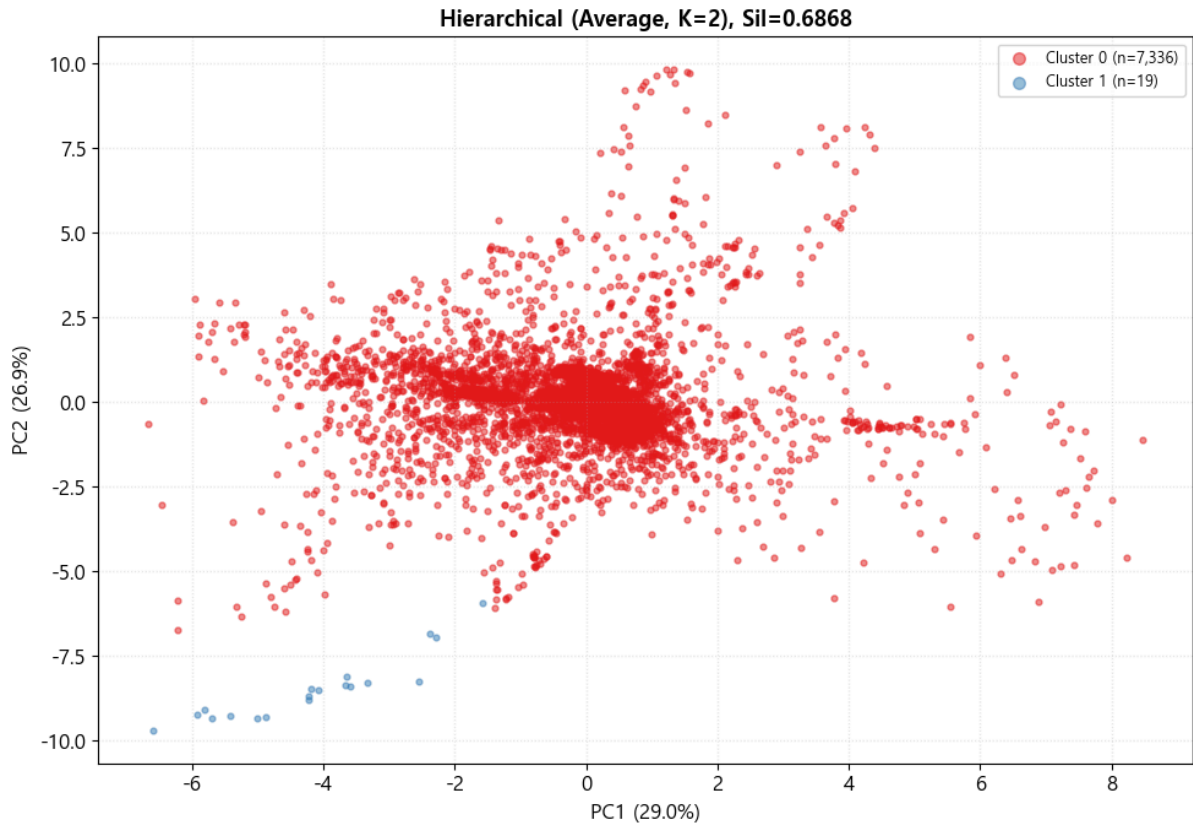
방법별 실루엣 스코어를 비교해본 결과 **Average**가 $k = 2$ 일때 가장 높았다.

Average Linkage는 두 군집간 모든 점 쌍의 평균 거리를 기준으로 병합한다. 이 데이터처럼 하나의 거대한 밀집 영역, 극소수 이상치로 구성된 경우, **Average**는 밀집 영역을 하나로 빠르게 통합하고 극단치만 분리하는 경향이 있다. 결론적으로 **DBSCAN**의 밀도 기반 관점과 가장 유사한 결과를 도출한다.

Ward는 분산 최소화 기준이라 **K-Means**와 유사한 결과를 낼것으로 예상했지만, $k = 4$ 에서 0.2234로 **K-Means**(0.3814)보다 더 낮았다.

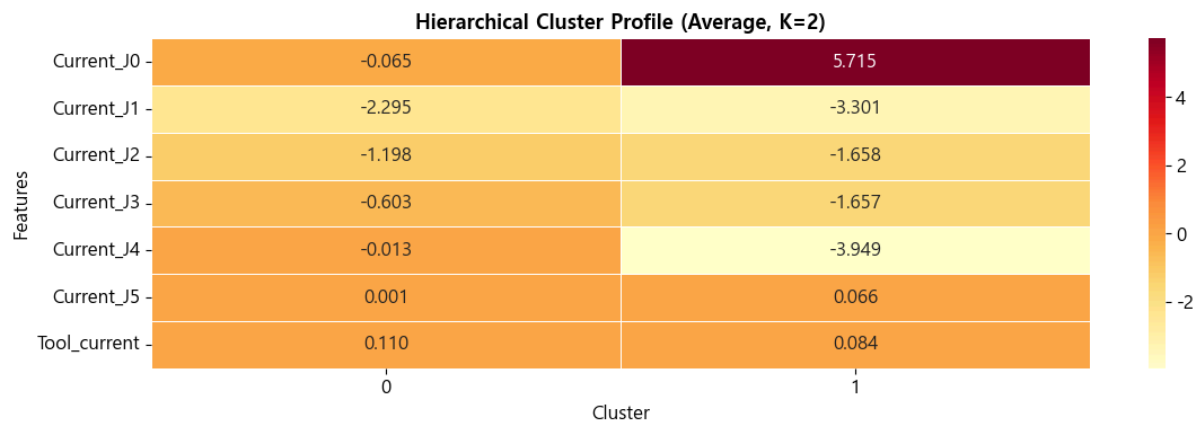
이유를 추측해보자면 **Ward**는 병합 과정에서 한번 합친 데이터를 되돌릴 수 없기 때문에 초기 단계의 잘못된 병합이 누적되어 $k = 4$ 시점에서 **K-Means**보다 경계가 흐려졌거나 로봇의 다각적 물리상태를 실제로 분할하고 있지만 군집 간 경계가 뚜렷하지 않아 실루엣 점수가 낮게 나온것으로 추측된다.

실루엣 점수가 가장 높았던 조건인 **Average, K=2**를 해석해보겠다.



```
[Hierarchical] Average Linkage, K=2
Silhouette Score: 0.6868
Cluster 0: 7,336개 (99.7%)
Cluster 1: 19개 (0.3%)
```

앞서 본 DBSCAN을 두개의 군집으로 나뉘던 결과와 거의 동일하다. 극단적 이상치 19개만 분리한것이다.



군집 히트맵을 분석한 결과, 마찬가지로 k = 2 DBSCAN과 거의 동일한 수치를 보였다.

```
=====
```

[교차표] K-Means vs Hierarchical

```
=====
```

Hierarchical	0	1	Total
KMeans			
0	5695	0	5695
1	294	0	294
2	1132	19	1151
3	215	0	215
Total	7336	19	7355

K-Means와 Average, K=2 Hierarchical을 비교해본 결과, Hierarchical Cluster 1의 19개는 K-Means Cluster 2(과부하, 1,151개)에서 가장 극단적인 19개만 뜯어낸 것이다.

K-Means의 Cluster 1(그리퍼), 3(특수케적)의 데이터는 단 한 개도 분리되지 않았다.

```
=====
```

[교차표] DBSCAN vs Hierarchical

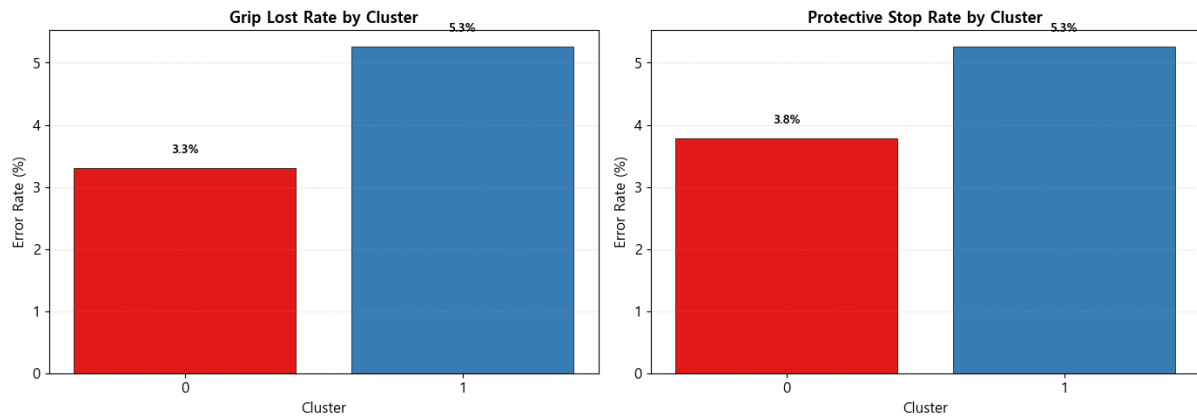
```
=====
```

Hierarchical	0	1	Total
DBSCAN			
-1	77	2	79
0	7231	0	7231
1	0	17	17
2	18	0	18
3	10	0	10
Total	7336	19	7355

k를 4로 조정해본 DBSCAN과 Average, K=2 Hierarchical을 비교해본 결과, DBSCAN Cluster 1의 17개가 Hierarchical Cluster 1의 19개에 완전 포함되는것을 확인할 수 있었다.

DBSCAN이 밀도 기반으로 찾은 극단적 이상치 17개가 Hierarchical이 병합 순서 기반으로 찾은 이상치 19개 안에 완전히 포함되어 있음을 의미한다.

두 알고리즘이 완전히 다른 원리로 접근했는데도 같은 19개를 가장 이질적인 데이터로 지목한 것은 이 데이터의 구조적 특성이 알고리즘에 독립적인 객관적 사실임을 강하게 시사한다.

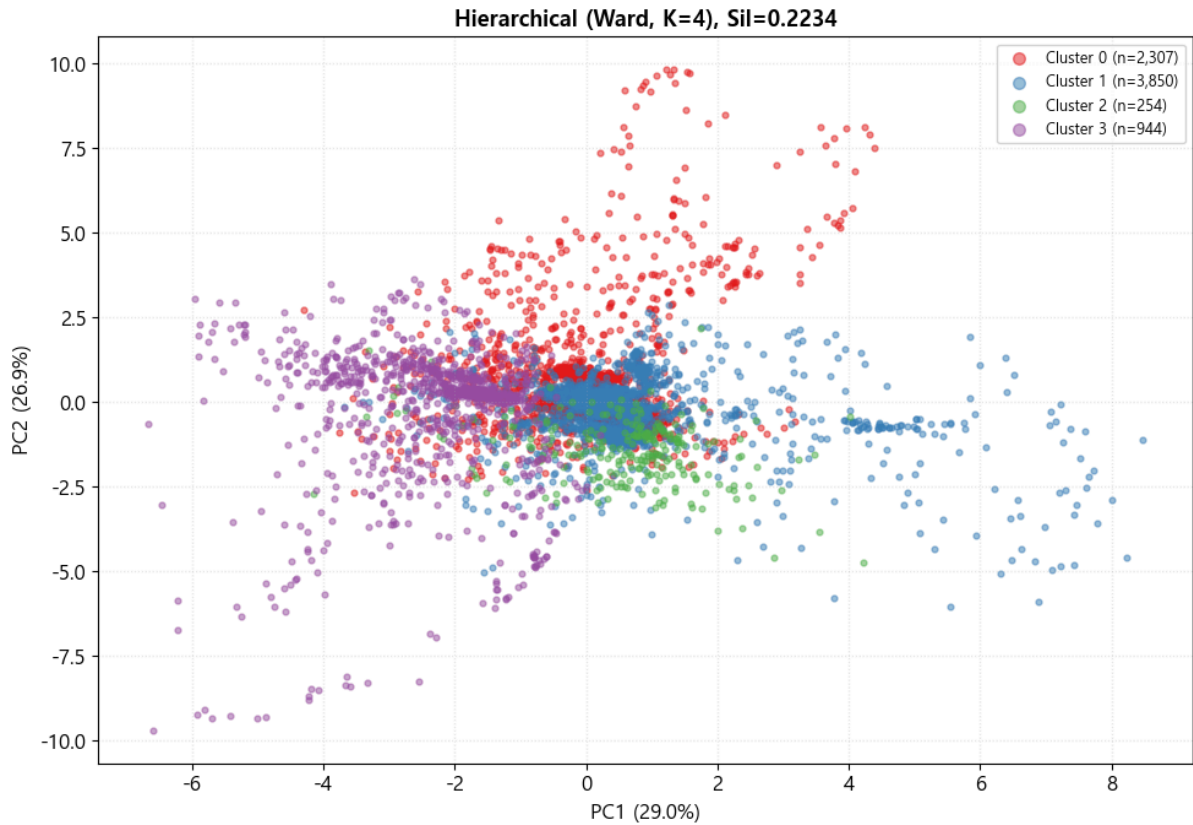


외부 검증을 해본 결과, 19개의 극단적 이상치에서 **Grip lost**와 **Protective Stop**가 모두 5.3% 정도 발생했다. 표본이 19개로 극소수이므로 통계적 의미보다는 방향성을 나타내는 정도로 해석된다.

앞서 **K-Means**와 직접 비교하기 위해 **Ward** 방법을 **k=4**로 맞추어 추가로 확인해본다.

Average K=2는 **DBSCAN**과 같은 관점의 반복으로 전체 데이터의 99% 이상을 하나의 거대 군집으로 묶고 극소수의 이상치만 분리해내어 수학적인 응집도 점수만 부풀려진 상태이며 로봇의 물리적 운영 상태를 세분화 하는 목적에는 잘 맞지 않다.

반면, **Ward k=4**는 **K-Means**와 같은 방식을 가지고 있어, 단순히 실루엣 점수를 극대화 하는 모델보다 앞선 **K-Means**와 동일하게 4개를 나누되, 병합 방식이 다를 때 결과가 얼마나 달라지는가를 확인해본다.

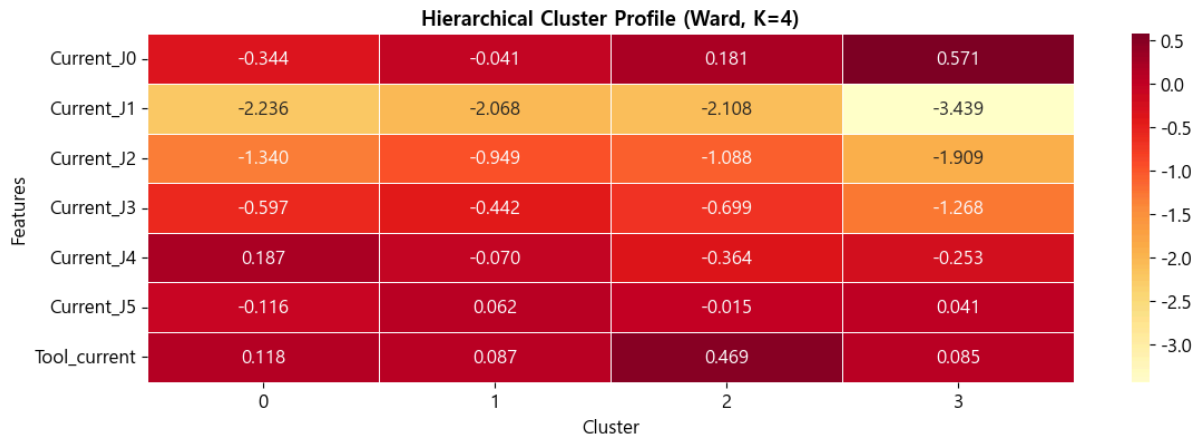


```
[Hierarchical] Ward Linkage, K=4  
Silhouette Score: 0.2234  
Cluster 0: 2,307개 (31.4%)  
Cluster 1: 3,850개 (52.3%)  
Cluster 2: 254개 (3.5%)  
Cluster 3: 944개 (12.8%)
```

K-Means의 군집 크기가 77%로 Cluster 0에 집중되었다면 Ward 방식은 Cluster 1이 52%로 최대이고 나머지도 더 높은 분포로 나뉘었다.

Ward는 분산을 최소화하기 위해 큰 군집을 더 적극적으로 쪼개는 경향이 있어서,

K-Means가 하나의 거대한 정상 데이터로 두었던 데이터를 두 덩이로 분리했다. 이것이 실루엣 스코어가 0.22수준으로 낮아진 직접적인 원인으로 추측된다.

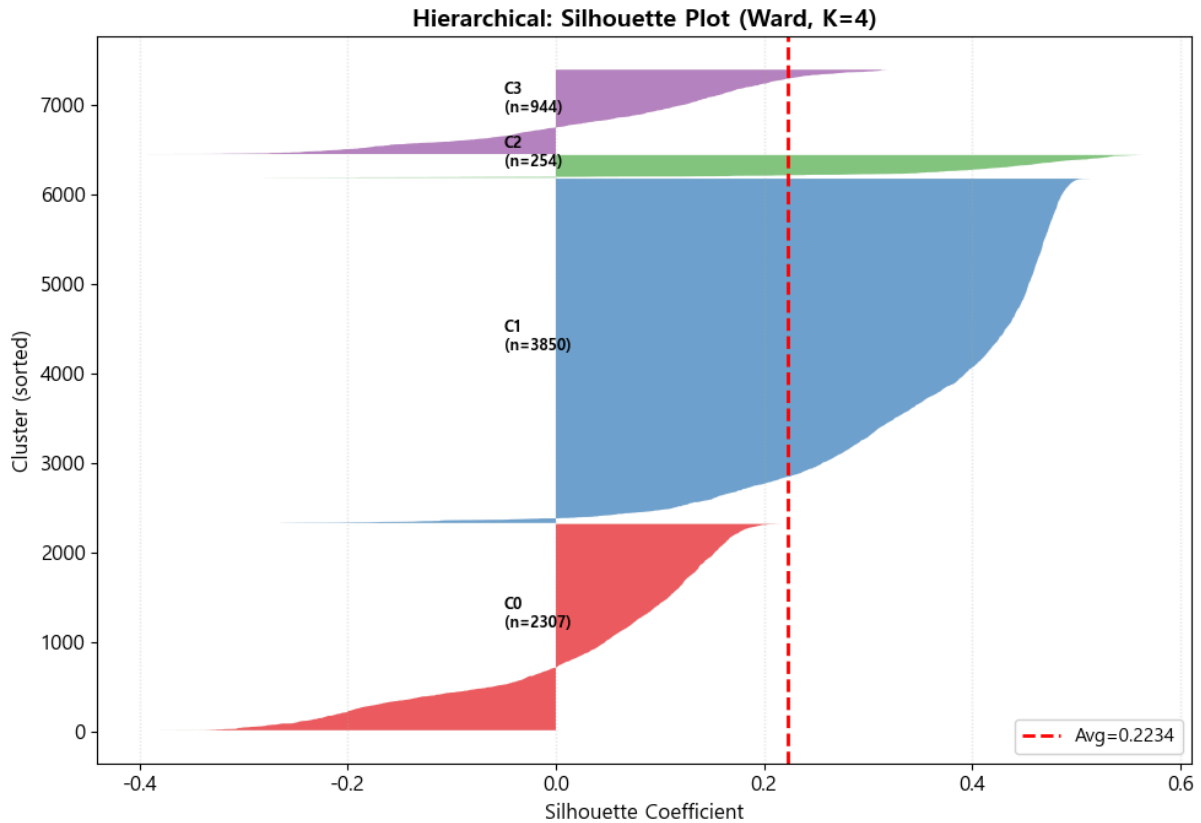


Ward, k=4인 군집 히트맵을 보면, **K-Means** 와 유사하지만 결정적인 차이가 있다.

Cluster 0과 **1**은 전반적인 전류 값들의 색상 분포가 비슷하다. 둘다 **J2**에 역방향 부하가 걸려있고 나머지 수치들은 극단적이지 않고 평이한 수치를 가진다.

Cluster 2는 맨 아래의 **Tool Current**가 0.4로 4개의 군집중 가장 높다. 교차표 분석에서 **K-Means**의 **Cluster 1** 데이터의 대부분이 이곳으로 넘어왔음을 확인했다. 로봇팔의 궤적은 험난하지 않지만, 끝단 그리퍼에만 강한 부하가 걸린 상태를 **K-Means**와 동일하게 잡아냈다.

Cluster 3은 **J1**과 **J2**에 다른 군집들보다 더 강한 역방향 부하가 걸려있다.



[군집별 Silhouette 통계]

```
Cluster 0: mean=0.0301, min=-0.3813, 음수비율=30.4%
Cluster 1: mean=0.3672, min=-0.2651, 음수비율=1.4%
Cluster 2: mean=0.3970, min=-0.2813, 음수비율=3.9%
Cluster 3: mean=0.0624, min=-0.3928, 음수비율=31.6%
```

실루엣 플롯을 보면, 전체 평균 실루엣 점수가 **K-Means**보다 떨어진 원인을 쉽게 알 수 있었다.

Cluster 0과 **3**을 보면, 음수 방향으로 깊숙히 파고든 영역이 크다. 데이터상으로 군집 **0**은 약 **30%**, 군집 **3**은 약 **32%**의 데이터가 음수 실루엣 값을 가진다. 즉, 군집 **0**과 군집 **3**에 속한 데이터 **3**개중 **1**개는 잘못된 군집에 갇혀있다는 뜻이다.

K-Means의 실루엣 플롯을 보면 음수 영역이 거의 없었다. **K-Means**는 매 반복마다 데이터를 더 가까운 중심점 쪽으로 재배치할 수 있기 때문이었다. 반면 계층적 군집화는 한번 병합해버리면 나중에 그것이 전체 군집 구조 관점에서 오판이었음이 드러나도 다시 떼어낼

수 없는 비가역적 특성을 가지기 때문에 초기의 선부른 병합이 누적되어 이 거대한 음수 영역으로 나타났음이 증명된다.

Cluster 1과 2는 오른쪽으로 두껍고 길게 뻗어있으며 평균선을 훌쩍 넘긴다. 정상 궤도를 주행하는 데이터들과 그리퍼가 더 힘을 쓰고있는 데이터들은 특징이 뚜렷하기에 다른 데이터와 섞이지 않고 강한 응집력을 유지했음을 시사한다.

```
=====
[교차표] K-Means vs Hierarchical
=====
```

Hierarchical	0	1	2	3	Total
KMeans					
0	1853	3734	6	102	5695
1	23	23	248	0	294
2	221	88	0	842	1151
3	210	5	0	0	215
Total	2307	3850	254	944	7355

ARI = 0.3529
NMI = 0.4246

```
=====
[교차표] DBSCAN vs Hierarchical
=====
```

Hierarchical	0	1	2	3	Total
DBSCAN					
-1	15	25	23	16	79
0	2282	3825	231	893	7231
1	0	0	0	17	17
2	0	0	0	18	18
3	10	0	0	0	10
Total	2307	3850	254	944	7355

ARI = 0.0180
NMI = 0.0278

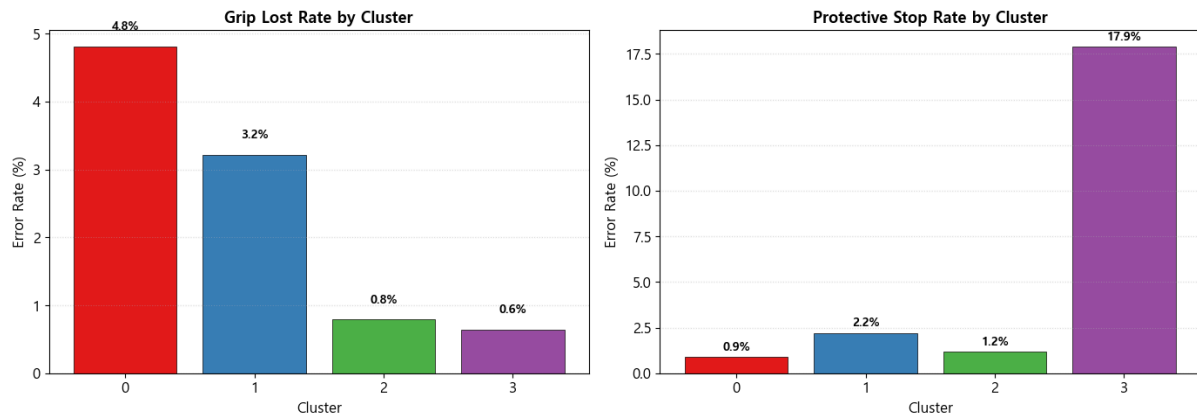
교차표를 보면 K-Means의 Cluster 1이 Hierarchical Cluster 2로 294개 중 248개(84%)가 일치한다.

K-Means의 Cluster 2이 Hierarchical Cluster 3으로 1151개 중 842개(73%)가 일치한다.

K-Means의 Cluster 0이 Hierarchical Cluster 0,1로 5695개가 1853, 3734개로 쪼개진것을 볼 수 있다.

ARI=0.3529, NMI=0.4246로 K-Means와 중간 수준의 일치도를 보인다.

DBSCAN보다 훨씬 높아, Ward가 K-Means와 같은 방향으로 데이터를 바라보고 있음을 보여준다.



```
=====
[외부 검증] Hierarchical (Ward, K=4) - 군집별 에러 발생률
=====
```

Cluster	Total	Grip Lost (%)	Protective Stop (%)
0	2307	4.81	0.91
1	3850	3.22	2.21
2	254	0.79	1.18
3	944	0.64	17.90

꽤 중요한 발견이다.

Cluster 3에서 Protective Stop 17.9%로 이것은 K-Means Cluster 2의 14.34%보다 높고 DBSCAN Cluster 2의 16.67%보다 높다.

Ward가 K-Means의 Cluster 2인 과부하 군집을 더 정밀하게 정제하여 에러 확률이 가장 높은 핵심 위험 데이터 944개만 골라내, 실루엣이 낮아도 에러 탐지력이 오히려 강해지는 결과를 얻었다.

따라서, 정리하면 Ward Linkage는 K-Means와 동일한 분산 최소화 철학을 공유하며,

ARI=0.35로 세 알고리즘 중 가장 높은 라벨 일치도를 보였다. 특히 K-Means가 과부하로

정의한 Cluster 2(1,151개)를 더 정밀하게 정제하여, Protective Stop 발생률 17.90%의

고위험 핵심 군집 944개를 도출하였다. 그러나 Silhouette 0.2234로 K-Means(0.3814)보다

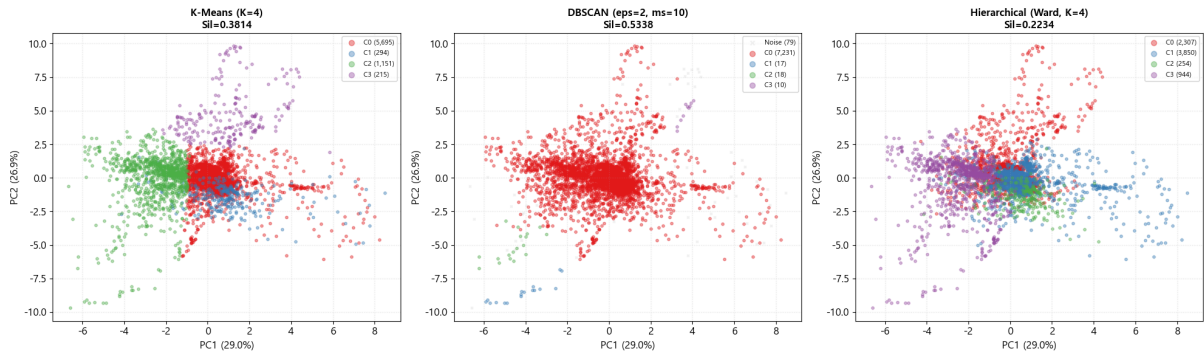
더 낮고 군집 0과 군집 3에서 30%가 넘는 데이터가 음수 값을 가지는 불안정한 구조를 보였다. 이는 계층적 병합의 비가역성으로 인해 정상 상태가 불필요하게 두 군집으로 분리되었기 때문이다. 따라서 로봇의 연속적이고 방대한 운영 상태를 4개의 대표 상태로 거시적 분류를 할 때는, 계층적 군집화보다 **K-Means** 알고리즘이 훨씬 안정적이고 유연한 모델임이 증명되었다.

6. 결론

본 분석에서는 UR3 Cobotops 데이터셋의 전류 피쳐 7개(Group A)를 기준으로 K-Means, DBSCAN, Hierarchical Clustering 세 가지 군집화 알고리즘을 적용하여 로봇의 운영 상태를 다각도로 분석하였다. 각 모델의 결과를 정리하면 다음과 같다.

평가 항목	K-Means (K=4)	DBSCAN (eps=2.0)	Hierarchical (Ward, K=4)
Silhouette	0.3814	0.5338*	0.2234
군집 수	4	4 (+ 노이즈 79개)	4
최대 군집 비율	77.4%	98.3%	52.3%
최고 Protective Stop	14.34% (C2)	16.67% (C2)	17.90% (C3)
핵심 강점	거시적 상태 분류	이상치 탐지	K-Means 교차 검증

*노이즈 제외 후 계산



세 알고리즘의 결과를 종합하면, 로봇의 운영 데이터는 불연속적으로 끊어지는 것이 아니라 연속적인 스펙트럼 위에서 점진적으로 변화하는 구조를 가진다는 것이 확인되었다.

DBSCAN과 Hierarchical(Average, K=2)이 독립적으로 하나의 거대한 덩어리와 극소수 이상치라는 동일한 결론에 도달한 것이 이를 보여준다. 이러한 연속적 데이터에서 K-Means만이 유일하게 의미 있는 4가지 운영 상태(정상 가동, 과부하, 그리퍼 하중, 특수 궤적)를 분리해냈으며, Ward Linkage가 ARI=0.35로 이 구조를 교차 검증하였다.

결론적으로, 각 알고리즘은 서로 다른 관점에서 상호 보완적인 역할을 수행한다.

K-Means는 로봇의 거시적 운영 상태를 4가지로 분류하는 주 분류기로서 가장 적합하다고 생각된다. DBSCAN은 밀도가 극단적으로 벗어난 데이터를 자동으로 탐지하는 이상치 탐지기로서의 역할을 할 수 있다. Hierarchical Clustering은 K-Means의 결과를 다른 알고리즘으로 교차 검증하는 역할을 수행하며, Ward 방식이 K-Means의 과부하 군집을 더 정밀하게 정제하여 Protective Stop 17.90%의 고위험 핵심 군집을 도출한 점은 주목할 만한 발견이다. 본 분석을 통해 실루엣 점수가 높다고 해서 반드시 좋은 군집화가 아니라는 점도 확인되었다. DBSCAN(0.5338)과 Hierarchical Average(0.6868)는 높은 실루엣을 기록했지만, 99% 이상을 하나의 군집에 몰아넣어 실질적인 상태 분류 기능은 하지 못했다. 반면 K-Means(0.3814)는 상대적으로 낮은 실루엣에도 불구하고, 물리적으로 해석 가능한 운영 상태를 성공적으로 도출하였다. 따라서 군집화 모델의 성능은 수학적 지표만이 아니라, 분석 목적에 부합하는 해석 가능성을 함께 고려하여 평가해야 함을 보여준다.